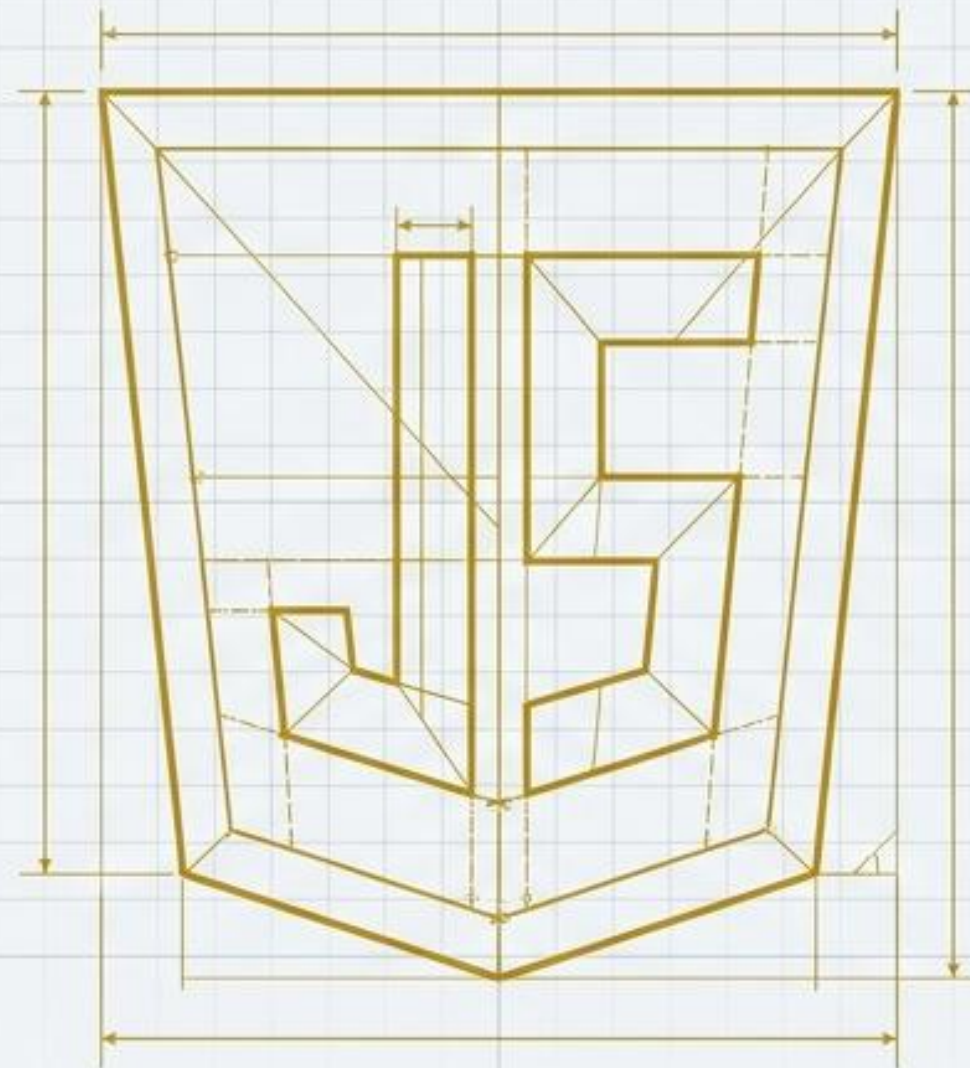




자바스크립트 핵심 가이드

기초 문법부터 실전 JSON 데이터 핸들링까지

> Runtime: Browser | Level: Essential | Focus: Data Pipeline

자바스크립트(JavaScript) 입문 핵심 요약 가이드

웹 브라우저의 등적 페이지 제작을 위한 객체 기반 스크립트 언어입니다.
ES6+ 변수 선언, 유연한 데이터 타입, 효율적인 데이터 처리의 올바른 사용법이 핵심입니다.

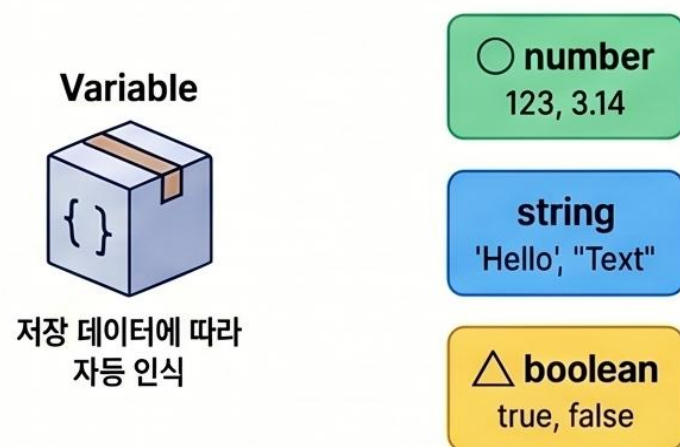
기초 정의 및 변수 선언 (Fundamentals & Variables)

현대적인 변수 선언 (ES6+)

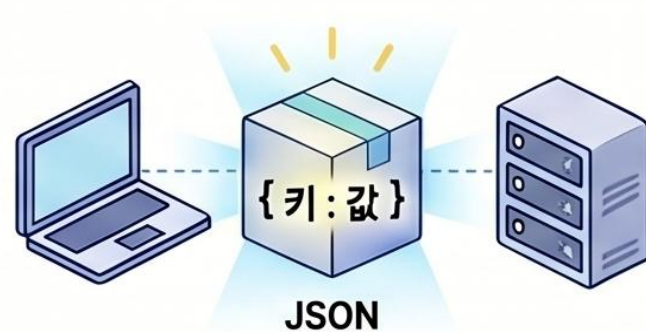


권장: 범위가 명확한 let과 상수를 의미하는 const 사용

유연한 데이터 타입 인식



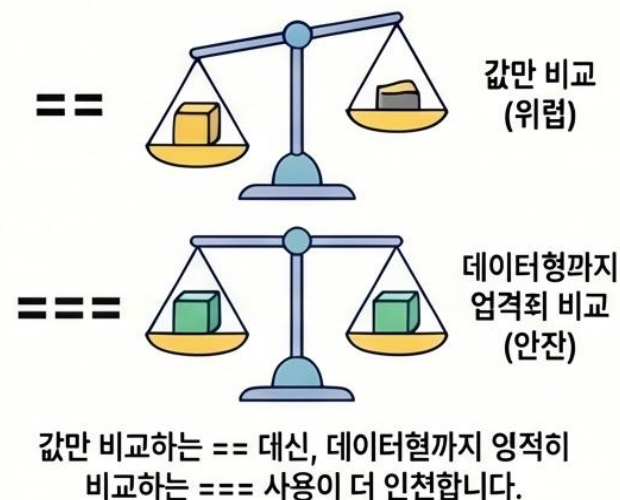
객체 표현의 표준, JSON



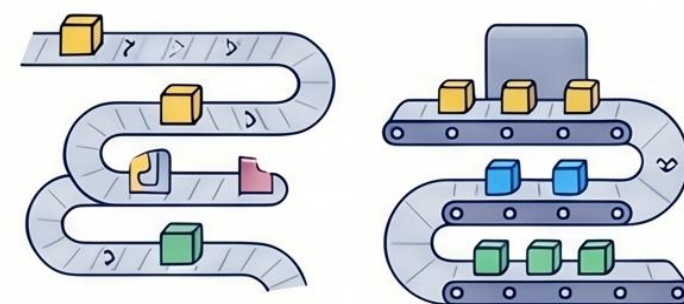
자바스크립트와 다른 시스템 간의
데이터 전송에 주로 사용

제어문 및 데이터 처리 (Logic & Data Flow)

일치 연산자(===) 사용 권장

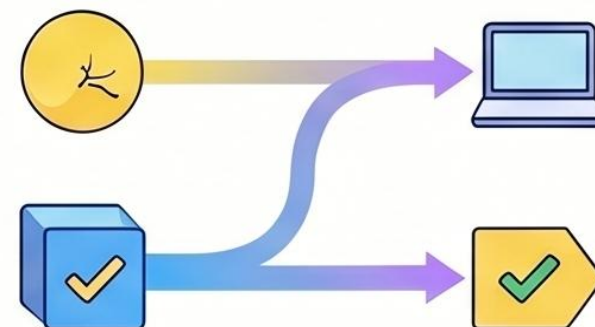


효율적인 배열 처리, map()과 forEach



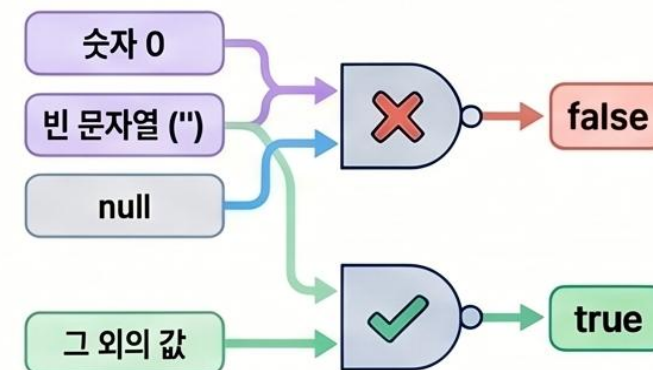
일반 for문보다 map이나 forEach 함수 사용이 권장됩니다.

논리형 변환의 특징, 금레



숫자 0, 빈 문자열(""), null 등은
false로 인식됩니다.

논리형 변환의 특징



그 외의 값은 true

웹 개발의 핵심, 자바스크립트(JavaScript) 기초 가이드

자바스크립트는 정적인 HTML을 동적으로 변화시키는 언어입니다. 본 가이드는 변수 선언, 데이터 타입, 효율적인 함수 작성법 및 브라우저의 태그를 직접 제어하는 DOM(문서 객체 모델)과 이벤트 처리의 핵심을 요약합니다.

핵심 문법과 데이터 구조

변수와 상수 (let vs const)

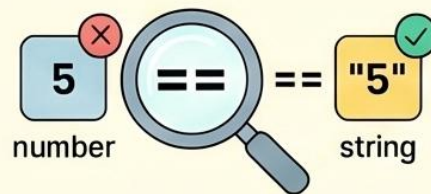


재할당이 가능한 지역 변수는 let을 사용합니다.



변경되지 않는 상수는 const를 사용합니다.

엄격한 비교 연산자 === 권장



값만 비교하는 == 대신, 데이터 타입까지 엄격히 비교하는 === 사용을 권장합니다.

데이터 관리 (Array & JSON)

```
[  
  Data 1,  
  Data 2,  
  Data 3,  
  ...  
]
```

배열 (Array)

[]는 배열, {}는 객체(JSON)를 의미하며 서버 데이터 통신 시 주로 사용됩니다.

```
{  
  Key 1: "Data 1",  
  Key 2: 123,  
  Key 3: "Data 2"  
}
```

객체 (JSON)

자바스크립트의 주요 데이터 형변환 함수 비교

Number() / parseInt()	String()	Boolean()
		
문자열을 숫자로 변환 Number('10') → 10	숫자를 문자열로 변환 String(10) → '10'	논리값으로 변환 Boolean(1) → true

함수 활용과 DOM 제어

화살표 함수 (Arrow Function)

```
function(x) {  
  return x * 2;  
}
```

기존 방식

function과 return 키워드를 생략한 간결한 코딩 방식을 권장합니다.

```
(x) => x * 2
```

권장 방식

DOM을 통한 HTML 태그 선택

```
<html>  
  <body>  
    <div>  
      <button id="myBtn">  
    </div>  
  </body>  
</html>
```

querySelector 또는 getElementById를 사용하여 HTML 요소를 선택하고 제어합니다.

사용자 이벤트 처리



onclick, onmouseover 등 사용자 명위에 반응하는 함수를 연결해 동적 기능을 구현합니다.

자바스크립트 핵심 가이드: 웹 데이터 처리와 필수 문법

브라우저 환경에서 자바스크립트(JS)의 역할과 서버(Java/Oracle) 데이터와의 연동 원리, 최신 문법 및 데이터 처리 기법 가이드.

웹 아키텍처와 데이터 관리의 기초

ES6+ 현대적 변수 선언 (let & const)

let

{ }

블록 스코프()로
메모리 효율적 관리

const

상수(const)
활용 권장

var

Deprecated

동적 데이터 타이핑

123

"ABC"

true

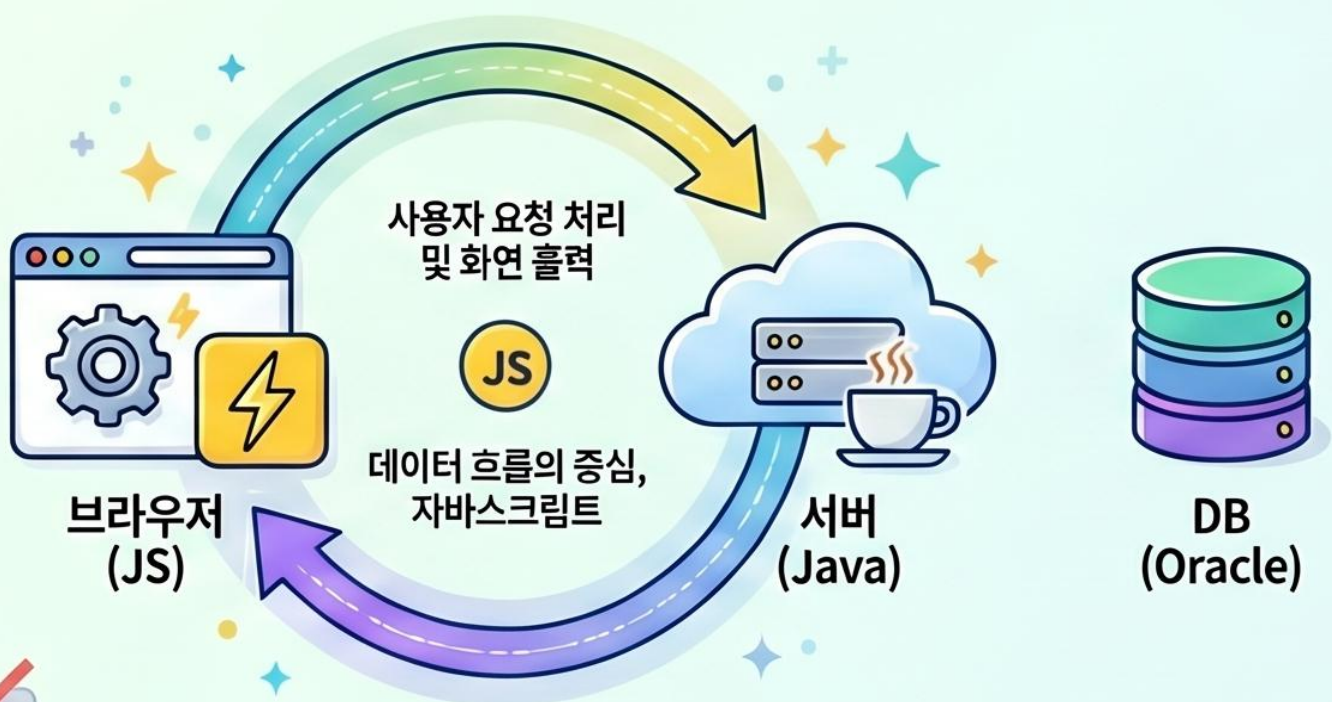
Number

String

Boolean

JS

별도의 데이터형 지정 없이
대입되는 값에 따라 Number,
String 등으로 자동 결정



자바스크립트의 주요 데이터형과 특징 요약

Number

10, 10.5

10, 10.5

정수와 실수를 구분하지 않고 숫자로 처리

JSON (Object)

{ }

{name:"홍길동"}

Java의 VO와 매핑되는 객체 모델

Array

[]

[1, 2, "A"]

다양한 형태의 데이터를 하나로 묶어 관리

데이터 매칭 및 효율적 출력

Java와 JS의 데이터 매칭

Java (서버)

List

VO 객체

JS (브라우저)

[] Array ()

{ } JSON ()

Java List ↔ JS Array, VO 객체 ↔ JSON 매장으로 데이터 전송

현대적인 반복문과 화살표 함수

[A, B, C]

=>

<p>A</p>

<p>B</p>

<p>C</p>

HTML 태그에 동적으로 데이터 삽입

HTML 데이터의 형변환 주의점

입력값 "10"

→

Number()

→

10

태그에서 읽어온 데이터는 기본 문자열, 산술 연산을 위해 Number() 변환 필수

자바스크립트(JS) 핵심 요약 노트: 기초부터 실전 제어까지

본 가이드는 자바스크립트의 구조화된 프로그래밍을 위한 함수의 다양한 선언 방식과 데이터를 효율적으로 관리하는 배열/객체의 활용법을 다룹니다.

Section 1: 유연한 함수와 효율적인 데이터 관리

세 가지 함수 선언 방식

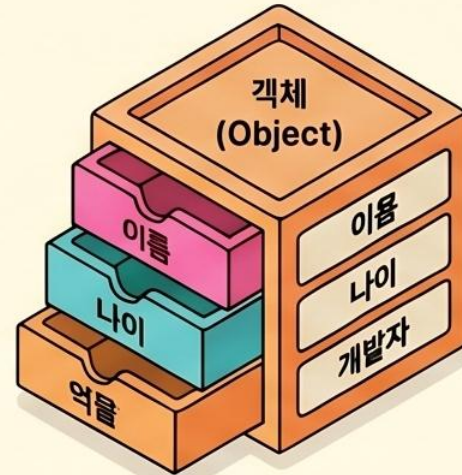
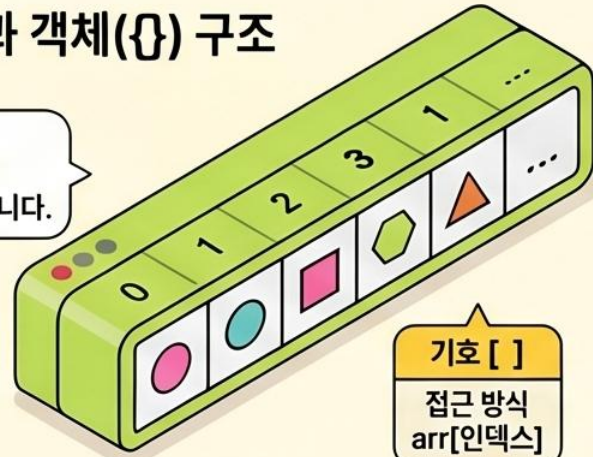
일반적인 선언적 함수 외에도 변수에 할당하는 익명 함수와 단결한 화살표 함수(=>)를 되짚어봅니다.



배열(Array)과 객체(Object) 구조

데이터 목록은 인덱스 기반의 배열로 정리합니다.

배열
(Array)

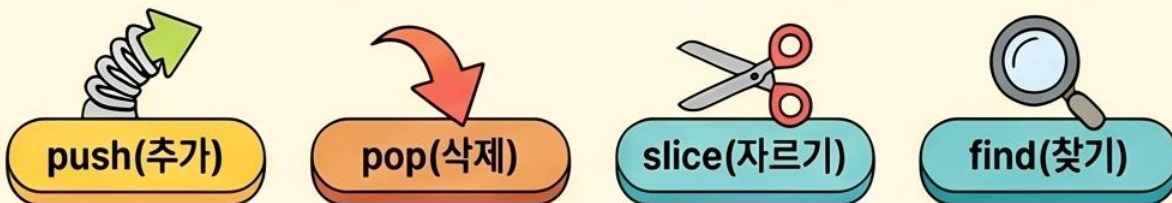


수성 중심의 데이터는 키(Key) 기반의 재제로 정리합니다.

기호 { }

접근 방식
obj.키 또는 obj['키']

주요 배열 조작 메서드

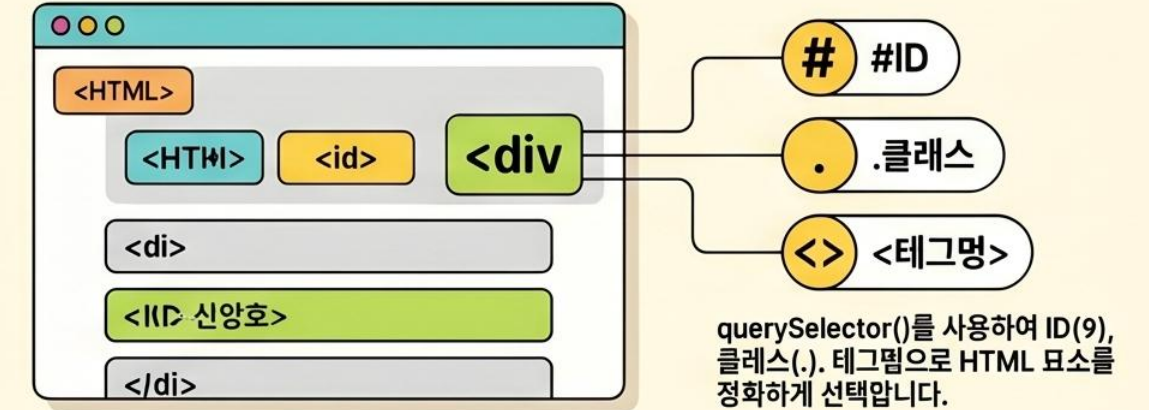


데이터를 유연하게 제어합니다.

배열과 객체 비교

구분	배열 (Array)	객체 (Object)
기호	[]	{ }
접근 방식	arr[인덱스]	obj.키 또는 obj['키']
주요 기능	순서가 있는 데이터 집합 (ArrayList)	범버 반수 중심의 데이터 모델 (VO)

Section 2: 동적인 웹 페이지 제어 (DOM & Event)



문서 객체(DOM) 조작

Before

```
<div>
  <div id="myDiv">기본 내용</div>
</div>
```

After

```
<div>
  <div id="myDiv">새로운 내용</div>
  style="color: blue;"
  data-status="active"
</div>
```

innerHTML로 내용을 변경하고, style 속성으로 CSS를 실시간 수정하며, 속성(attr)을 제어합니다.

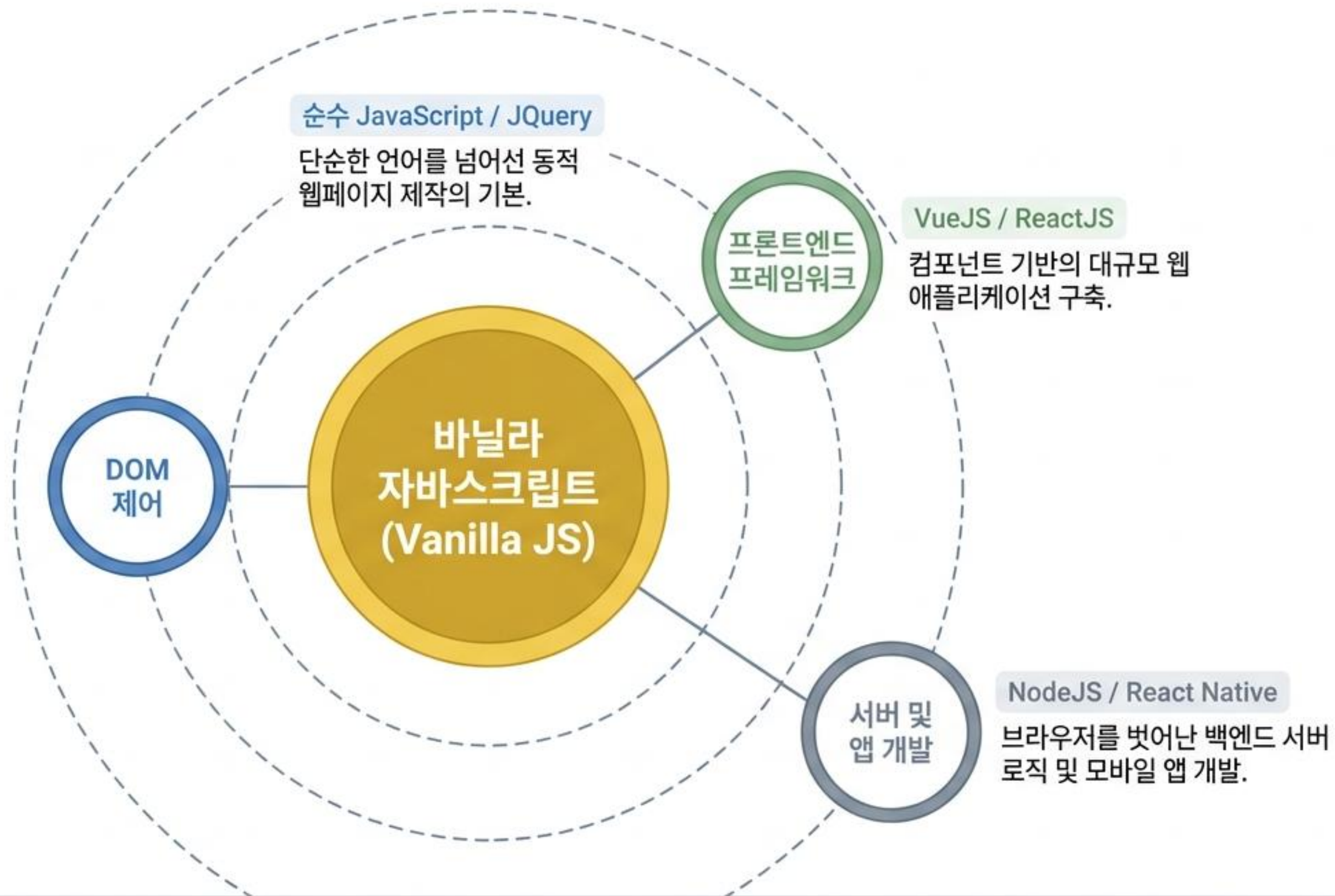
사용자 이벤트 반응(Listener)

고전적 방식



현대적 방식 (addEventListener)





자바스크립트는 단순한 웹 스크립트 언어를 넘어, 프론트엔드와 백엔드, 모바일을 아우르는 통합 생태계입니다.

언어의 3대 특징



인터프리터 언어

컴파일 과정 없이
브라우저 환경에서
즉시 한 줄씩 번역되고
번역되고 실행됩니다.



객체 기반

모든 데이터와 기능이
JSON(JavaScript Object
Notation) 포맷 중심의
객체로 구성됩니다.



이벤트 중심

사용자의 상호작용(클릭,
스크롤 등)과 시스템의
특정 상태 변화에
반응하여 동작합니다.

변수 선언의 진화

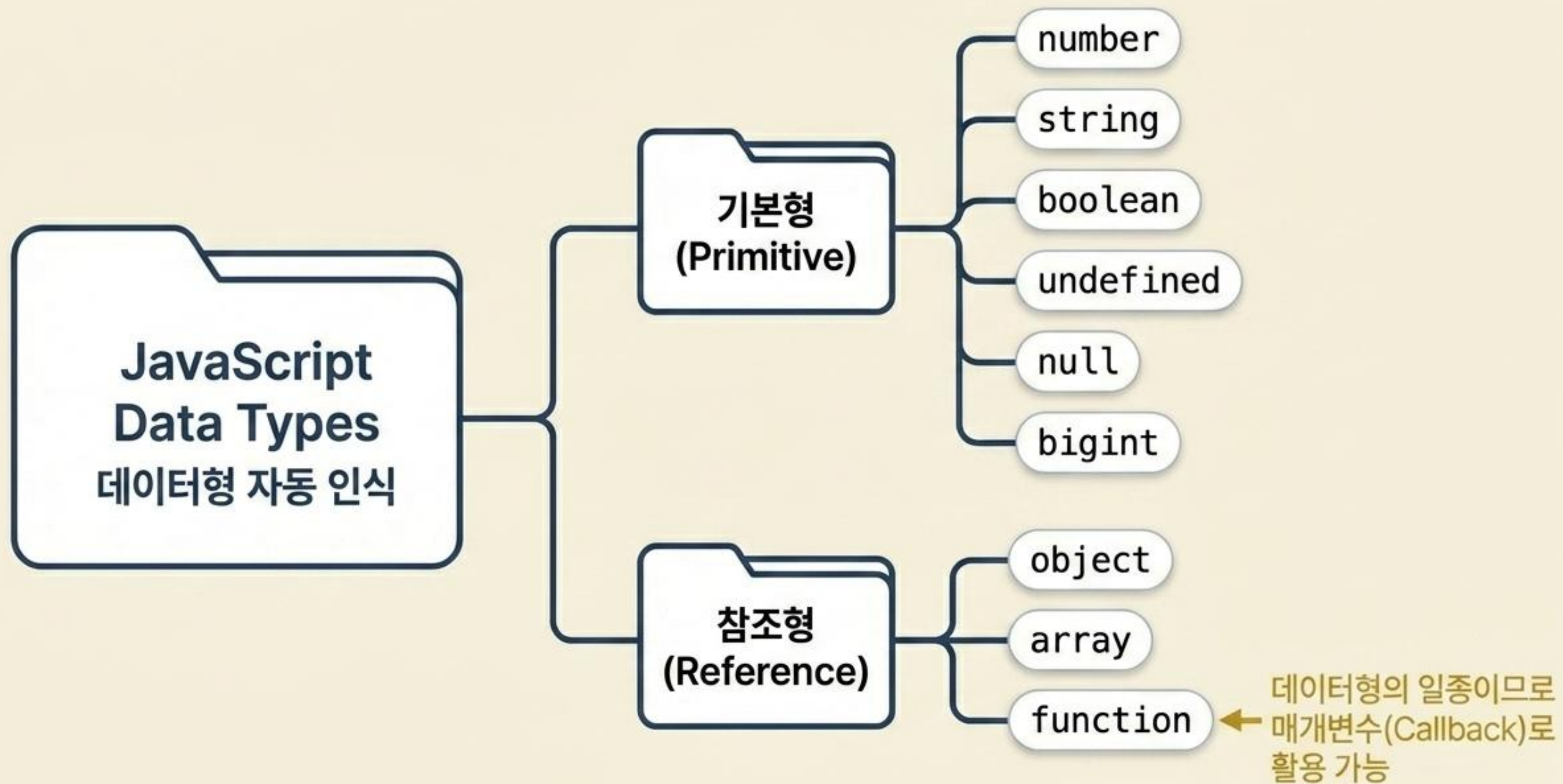
	var (ES5)	let (ES6)	const (ES6)
재선언	 스코프 모호성		
데이터 변경 (재할당)			 상수형 / 고정

```
let a = 10;  
a = 'Hello';  
a = [];  
a = {};
```

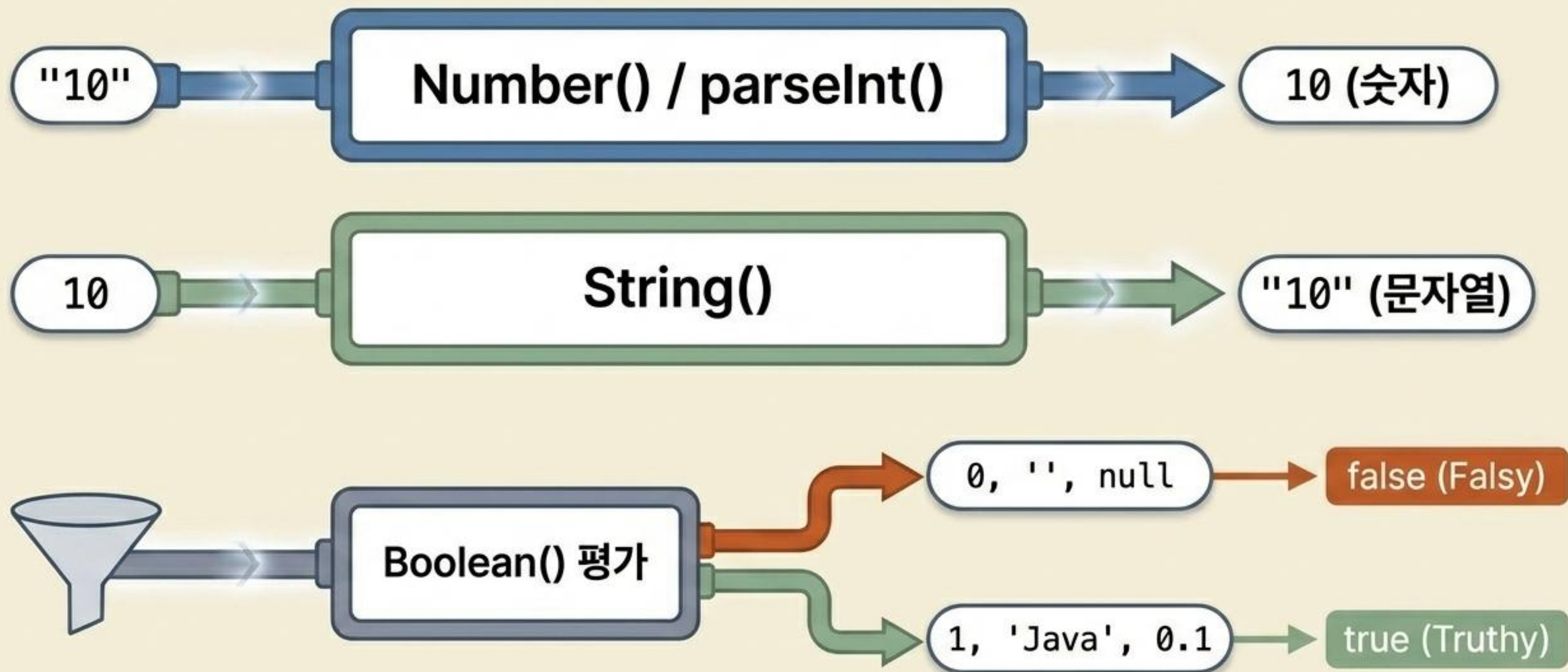
가독성 저하 위험!
데이터형을 고정하려면
TypeScript 도입 필요.

권장: 가독성과 안정성을 위해 ES6 표준인 let과 const를 사용하십시오.

데이터 타입 트리



형변환과 논리 평가



핵심 연산자 비교

function

cons

t)

'10' == 10 -> true

값만 비교 (데이터형 무시)

'10' === 10 -> false

값과 데이터형 완벽 일치 비교

t')

}

error

}



경고: 항상 === 및 !== 사용을 권장합니다!

&& (직렬/교집합)

양쪽 조건이 모두 true일 때만 true (범위/기간 포함)

|| (병렬/합집합)

양쪽 중 하나라도 true면 true (범위 이탈)

조건 ? true값 : false값

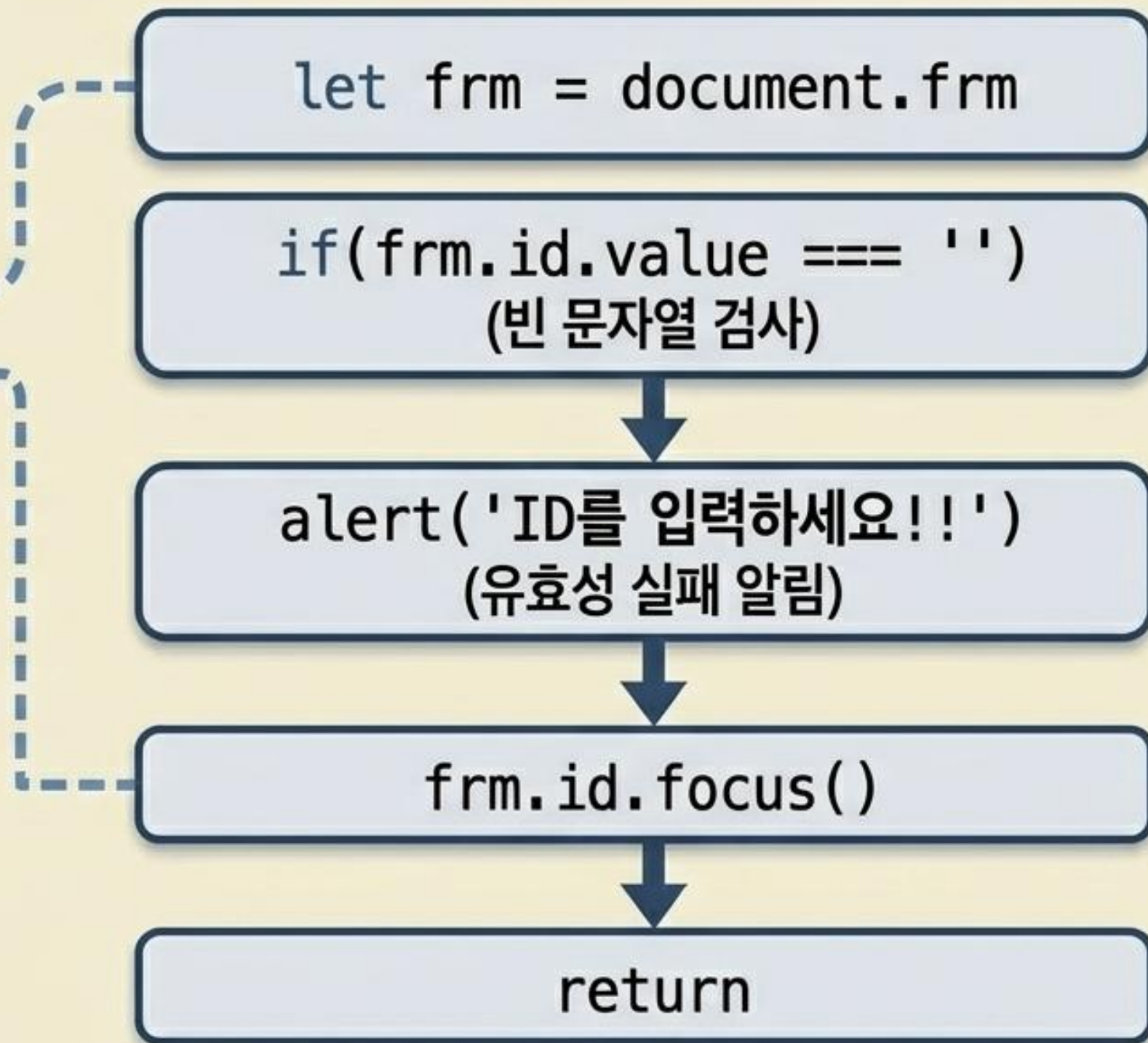
if-else문을 완벽히 대체하는 삼항 연산자

제어 로직과 DOM 제어 매핑

ID

Password

로그인



반복문의 진화 스펙트럼

고전적 제어 (Imperative)

`while / do~while`

`for (i=1; i<=10; i++)`

반복 횟수나 조건이 명확할 때 사용

컬렉션 순회 (ES6)

`for-in`

인덱스/키 순회

`for-of`

실제 데이터 순회

함수형 순회 (Declarative)

`forEach( (item) => {} )`

단순 데이터 순회 및 실행

`map( (item, index) => {} )`

순회 후 새로운 배열 반환 (가장 권장됨)

자바스크립트의 핵심 데이터 구조

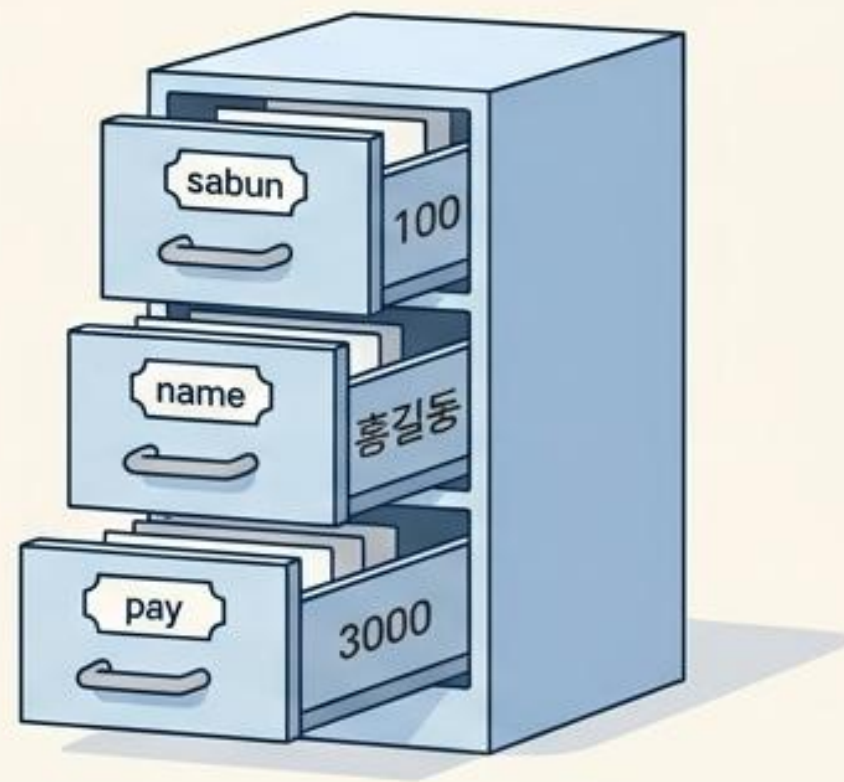
배열 (Array)



```
const data = ['홍길동', 20, true, '서울'];
```

서로 다른 데이터형을 혼합하여
순차적(인덱스)으로 저장하는 구조.

객체 (Object)



```
const sawon = {'sabun': 100,  
               'name': '홍길동', 'pay': 3000};
```

키(멤버변수)와 값(초기값)의
쌍으로 구성된 데이터 블록.

이기종 간의 데이터 브릿지



백엔드의 List<VO> 구조는 네트워크를 건널 때 JSON 문자열로 변환되며,
브라우저에 도착하면 자바스크립트의 객체 배열 형태로 즉시 파싱되어 화면에 렌더링됩니다.

실전 응용: JSON 파싱과 map() 활용

```
{  
  'movieNm': '극장판 체인소 맨: 레제편', 'genre': '애니메이션,액션,어드벤처', 'openDt': '2023-10-25',  
  'movieNm': '보스', 'genre': '코미디,액션', 'openDt': '2023-11-01',  
  'movieNm': '검자검 고장', 'genre': '에이,요스', 'openDt': '2023-10-20',  
  'movieNm': '기로나던', 'genre': '코미디,액션', 'openDt': '2023-11-25',  
  'movieNm': '상감농강', 'genre': '코미디,액션', 'openDt': '2023-11-01',  
  'movieNm': '언불달사', 'genre': '한천 왜선태인', 'openDt': '2023-11-03'  
}
```

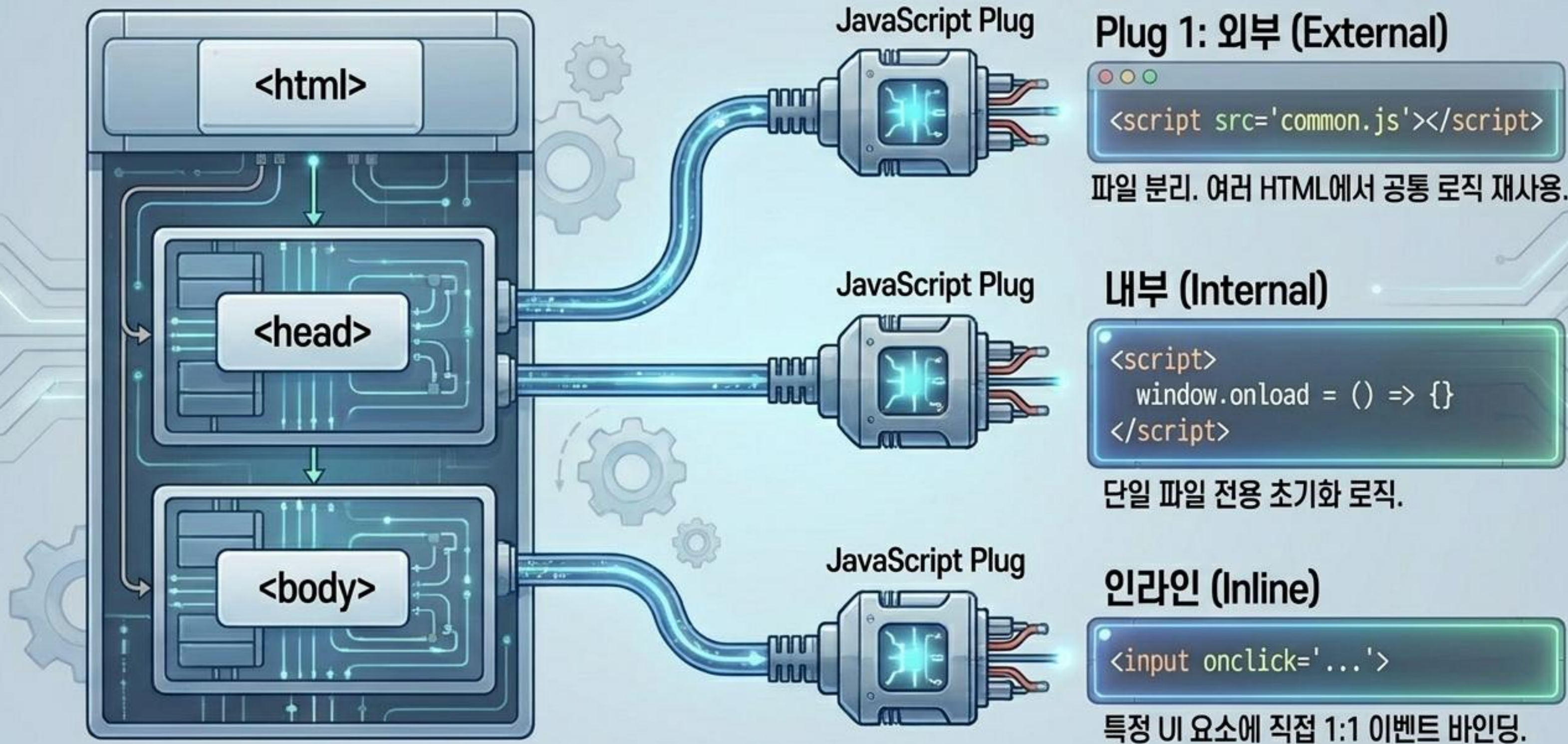
movieNm: 극장판 체인소 맨: 레제편

genre: 애니메이션

```
movies.map((movie, index) => {  
  document.write((index+1) + '.' +  
    movie.movieNm + '(' + movie.genre + ')')  
});
```

map() 함수를 사용하여 복잡한 JSON 배열 안을 순회하며 필요한 Key-Value 쌍만 정확히 추출해 DOM에 렌더링합니다.

브라우저 통합 및 이벤트 매핑



모던 자바스크립트 핵심 요약 (Cheat Sheet)

01. 변수 선언 (Variables)

`var`는 잊으세요. 데이터는 `let`으로, 고정값은 `const`로 선언하십시오.

02. 논리 비교 (Strict Equality)

타입 자동 변환의 함정을 피하기 위해 비교는 반드시 `===`를 사용하십시오.

03. 데이터 순회 (Functional Iteration)

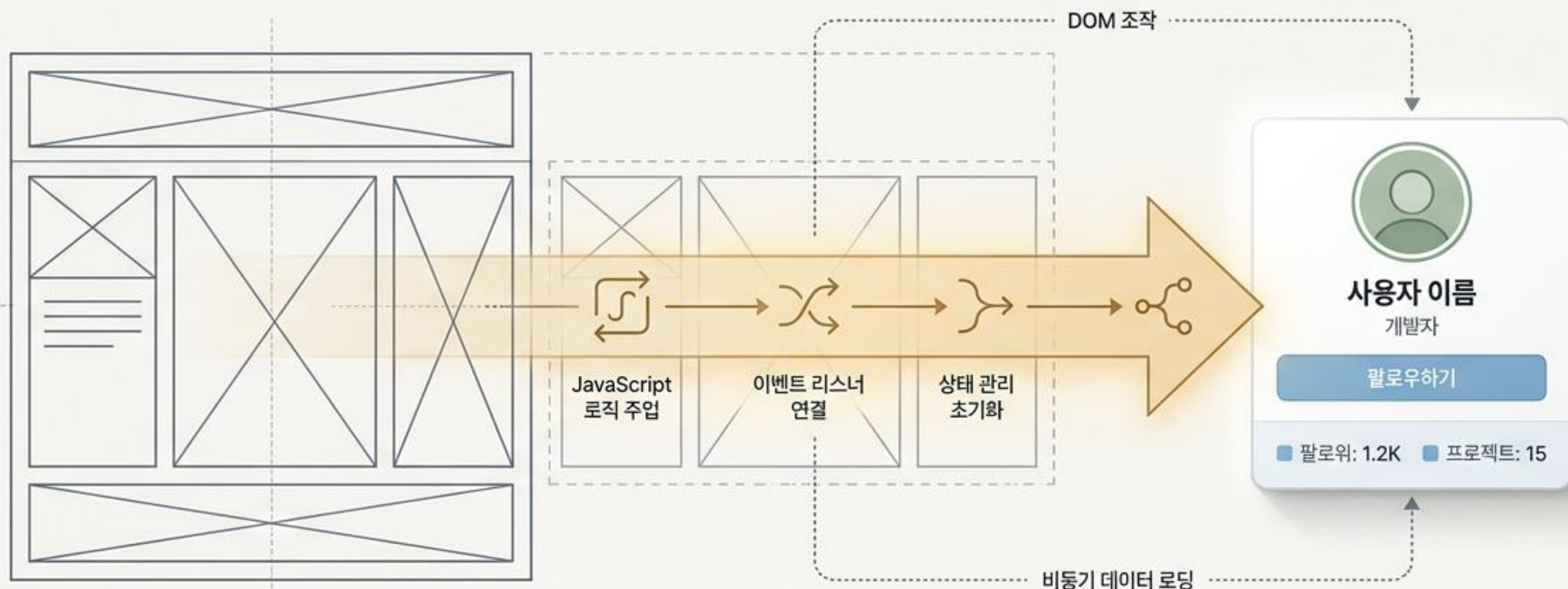
고전적인 `for`문 대신, 배열과 객체의 순회는 `.map()`과 `.forEach()`를 적극 활용하십시오.

04. 데이터 통신 (The Universal Format)

화살표 함수 `( () => {} )`를 통해 로직을 간결하게 유지하고, 모든 외부 데이터 통신은 `[ {}, {}, {} ]` 형태의 JSON 파이프라인으로 귀결됨을 기억하십시오.

정적 웹에서 동적 웹으로: 자바스크립트 코어 플레이북

웹페이지에 생명력을 불어넣는 4가지 핵심 원리와 실전 코드 스니펫



정적 웹: HTML 스펠레톤 (Before)

동적 웹: 생동감 넘치는 UI (After)

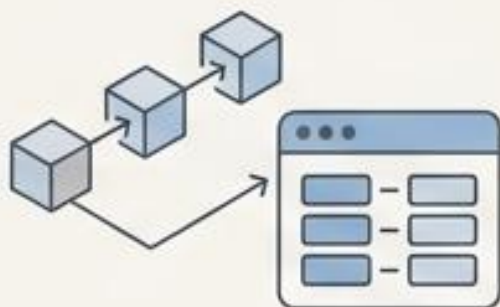
자바스크립트를 지탱하는 4개의 기둥



함수 (Functions)

재사용 가능한 명령문의 집합.
애플리케이션의 두뇌 역할.

```
function init() {  
  ...  
}
```



자료구조 (Arrays & Objects)

정보를 순서대로, 혹은
키(Key) 값으로 담아두는 그릇.

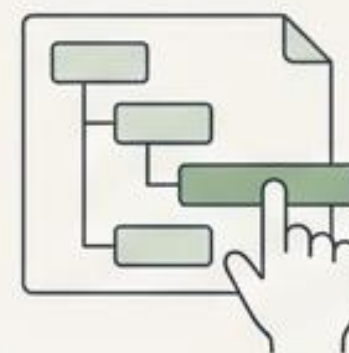
```
const data = { id: 1, ... };
```



이벤트 (Events)

클릭, 마우스 오버 등
사용자의 행동을 감지하는 센서.

```
element.addEventListener('click', ...)
```



DOM 제어 (DOM Manipulation)

HTML 뼈대를 찾고, 스타일과
내용을 동적으로 조작하는 손발.

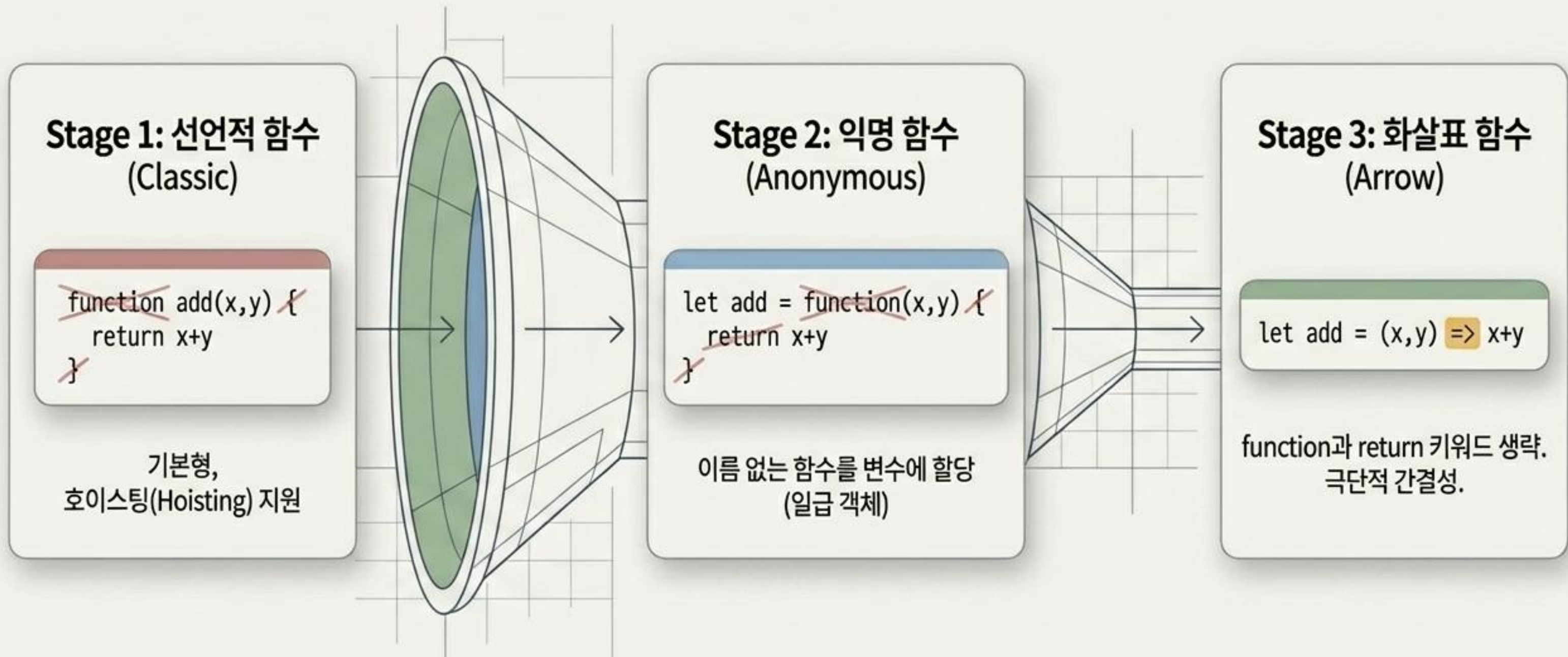
```
document.querySelector('.main').style...
```


함수의 해부학: 입력과 출력의 2x2 매트릭스

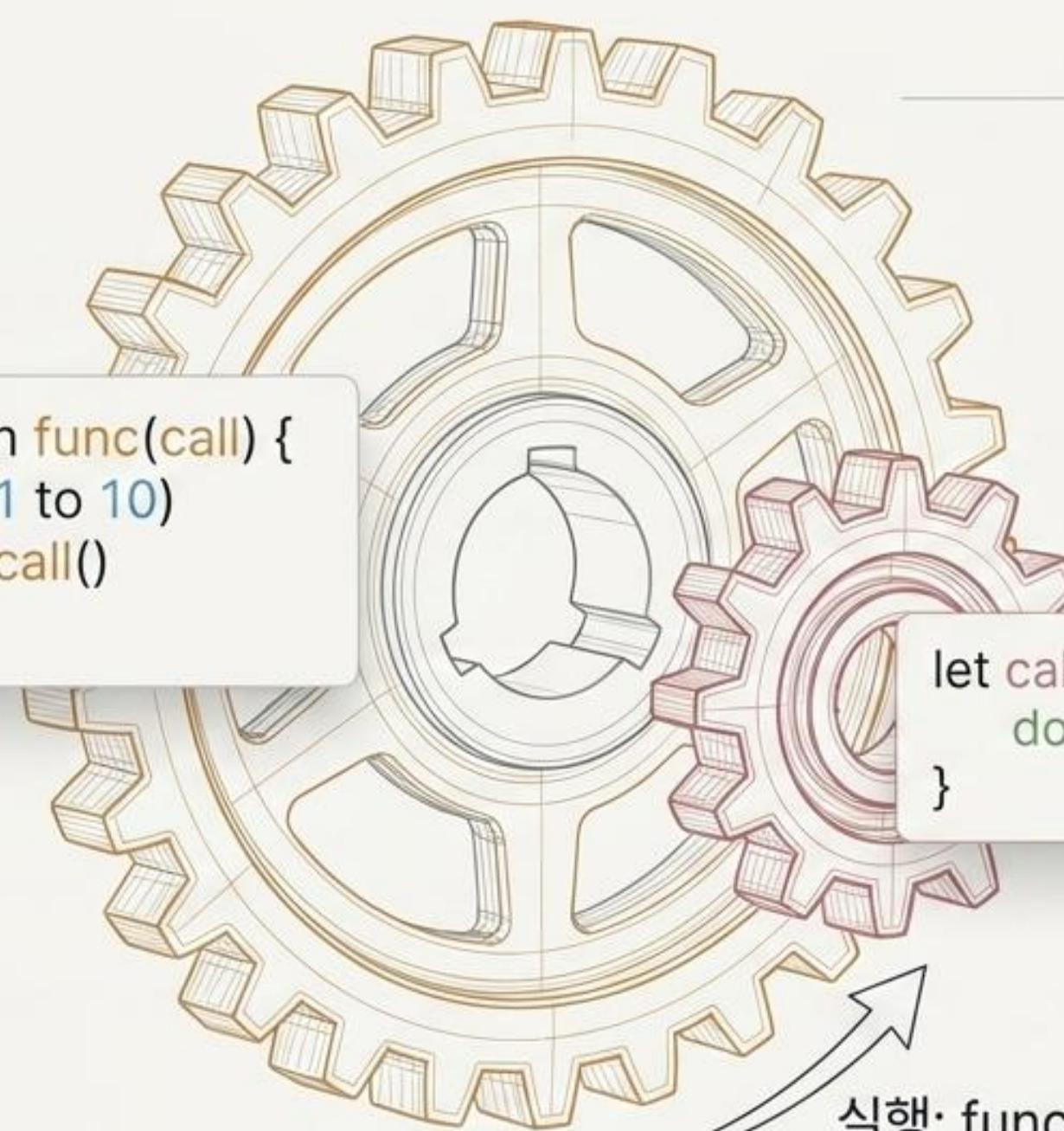
		매개변수 (Parameters) O/X	
		O	X
리턴형 (Return) O/X	O	<pre>function plus(a,b) { return a+b }</pre> 데이터 가공 후 반환	<pre>function getPi() { return 3.14 }</pre> 고정 데이터 반환
	X	<pre>function setAlert(msg) { alert(msg) }</pre> 입력값 기반 단순 실행	<pre>function init() { alert('시작') }</pre> 단순 반복 작업 처리

핵심 인사이트: 자바스크립트는 function을 독립적인 데이터형으로 인식하여 변수처럼 취급합니다.

문법의 진화: 함수 다이어트 (Syntactic Sugar)



콜백 (Callback): 매개변수가 된 함수



```
function func(call) {  
  for(1 to 10)  
    call()  
}
```

```
let callback = function() {  
  document.write('호출')  
}
```

실행: func(callback)

정의:

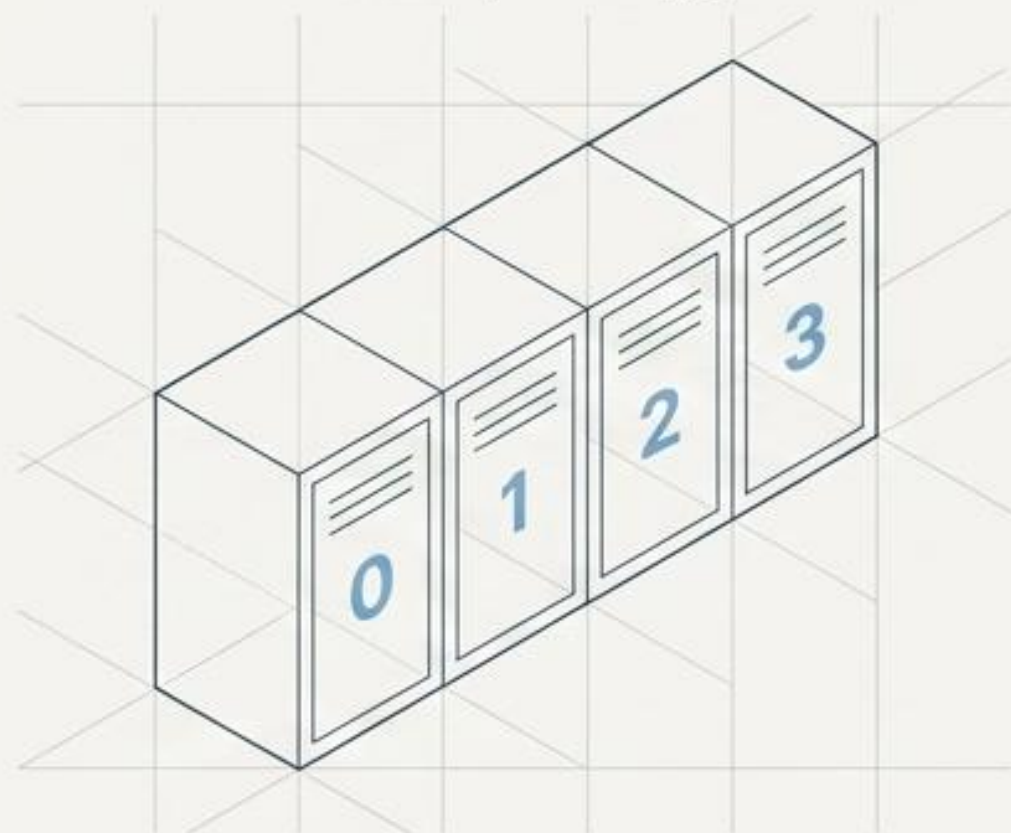
다른 함수의 매개변수로 전달되어, 특정 시점에 호출되는 함수.

활용처:

setTimeout(callback), 이벤트 리스너, JQuery, 비동기 데이터 처리(React-Query) 등 모던 웹의 핵심 메커니즘.

데이터를 담는 두 가지 그릇: 배열 vs 객체

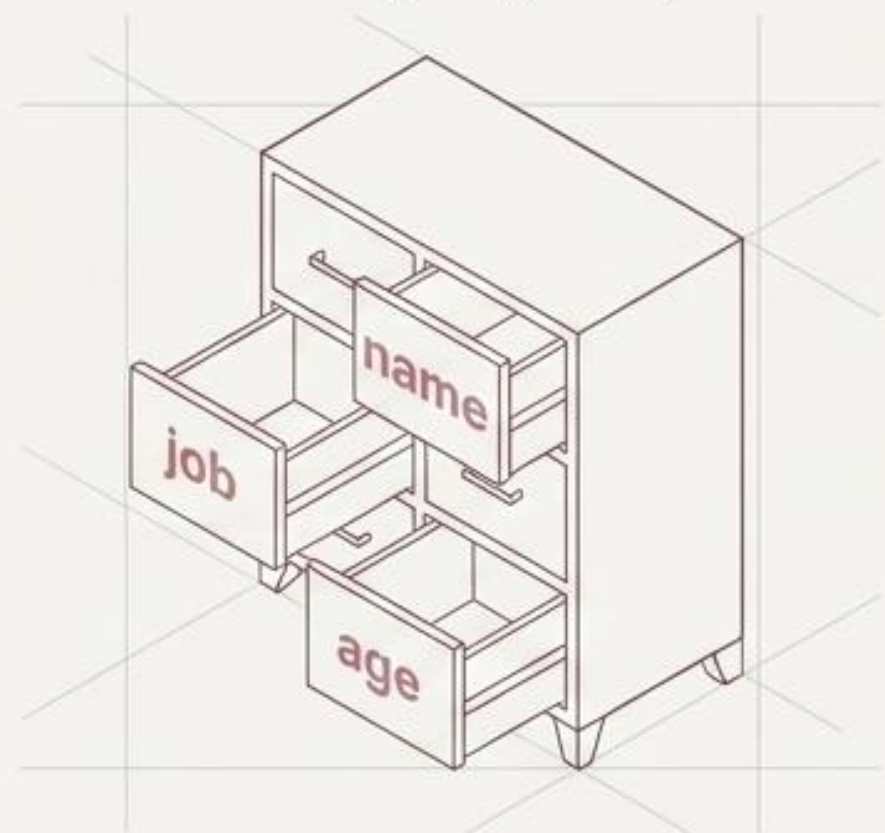
배열 (Array)



```
let arr = ['홍길동', '심청이']
```

핵심 특징: 순서(Index) 중심.
순차적인 데이터 목록 관리에 최적.

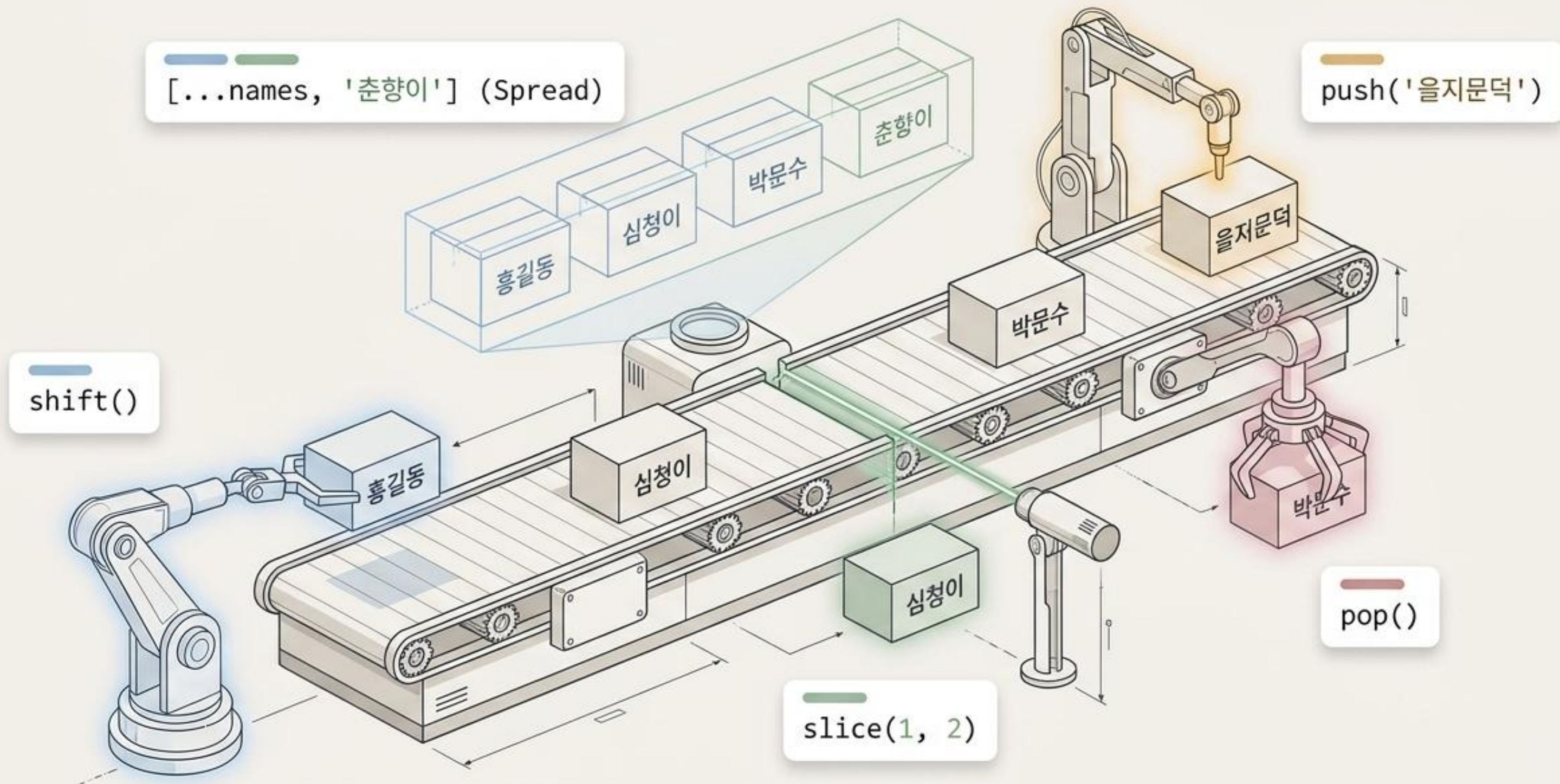
객체 (Object)



```
let obj = { name: '홍길동', job: '대리' }
```

핵심 특징: 키-값(Key-Value) 중심.
데이터의 의미(이름)를 명확히 부여할 때 최적.

배열 조작 메커니즘 (Array Methods)



객체의 활용: 데이터에 이름표 붙이기

```
let sawon = {  
  sabun: 1,  
  name: '홍길동',  
  dept: '개발부'  
}
```

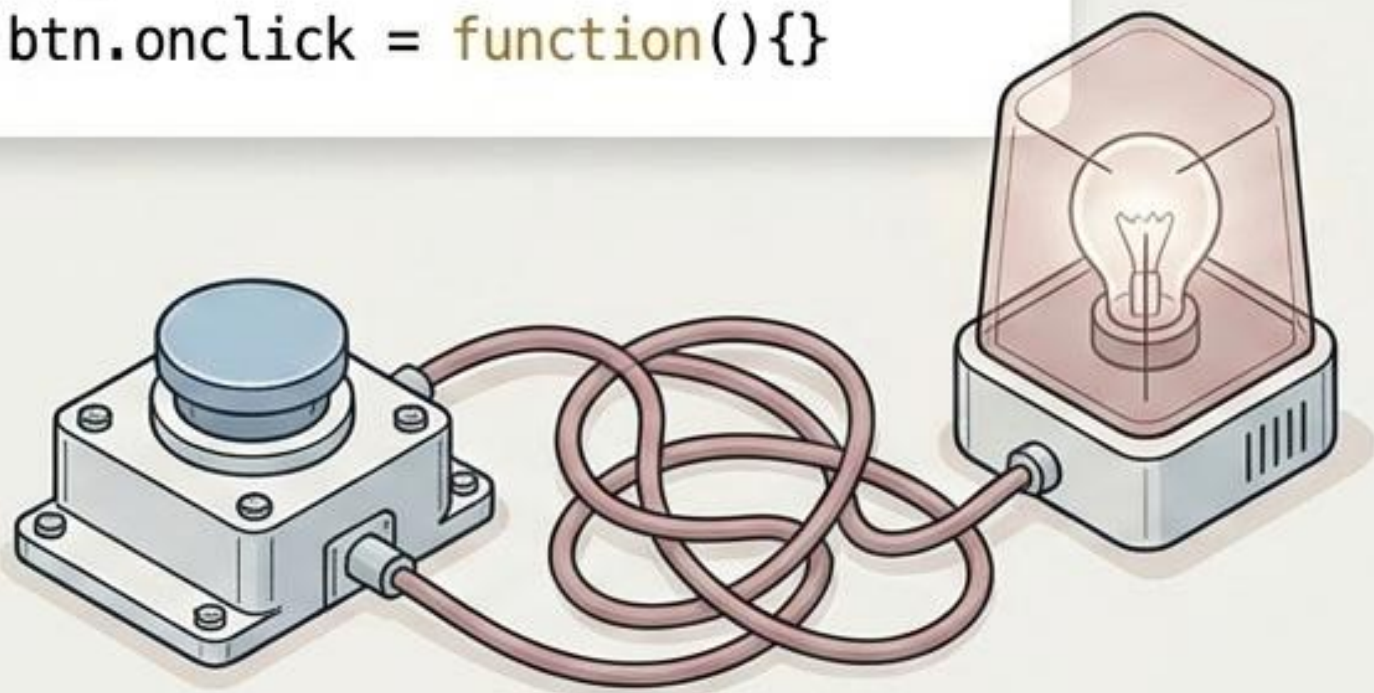


데이터 순회 마스터키: `for(let key in sawon)`
객체의 모든 속성을 순회하며 데이터를 추출합니다.

이벤트 핸들링: 행동을 감지하는 센서

고전적 모델 (Classic / Inline)

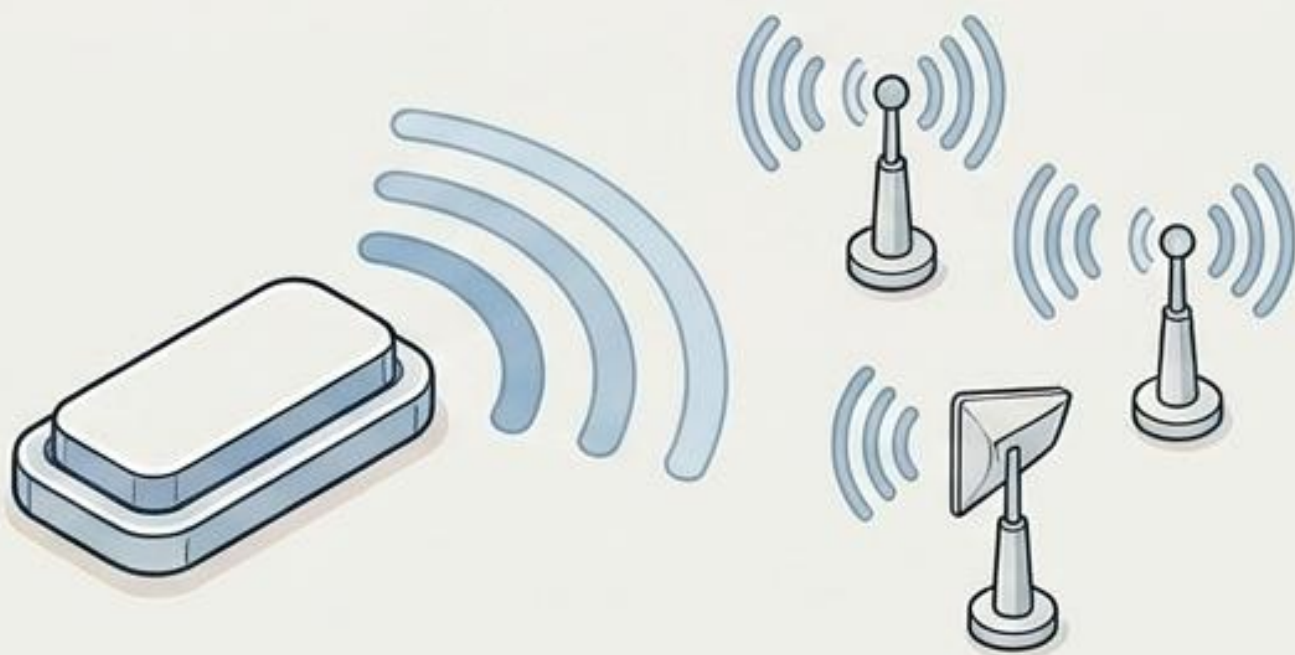
```
<button onclick='alert("클릭")'>  
또는  
btn.onclick = function(){}
```



한계점: 하나의 이벤트에 하나의 동작만 연결 가능.
HTML과 JS가 강하게 결합됨.

모던 리스너 (Modern Listener)

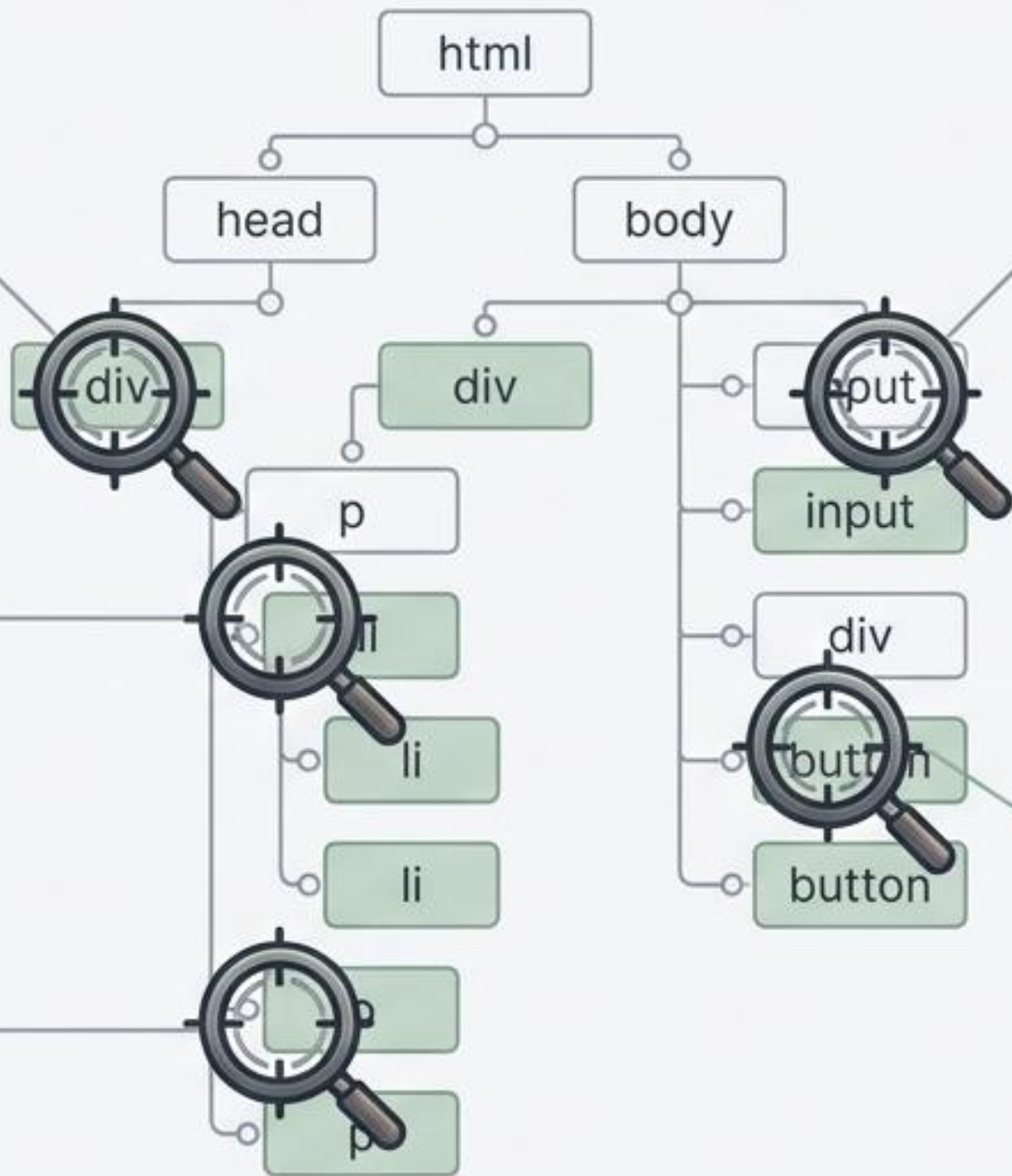
```
btn.addEventListener('click', function(){})
```



장점: 다중 이벤트 등록 가능.
HTML과 JS의 완벽한 분리. 모던 웹의 표준.

DOM 셀렉터: 대상을 낚아채는 5가지 무기

HTML DOM Tree



getElementById('id명')

`<div nid="#uniqueId">`
단 하나의 고유한 노드를 정밀하게 타겟팅.

getElementsByClassName('class명')

`<li class=".item">`
동일한 클래스를 가진 요소들을 컬렉션으로 포획.

getElementsByTagName('태그명')

`<p nag=</p>`
특정 종류의 모든 태그(예: img)를 일괄 수집.

getElementsByName('name명')

`<input nam="name="user" >`
주로 Form 요소의 name 속성으로 타겟팅.

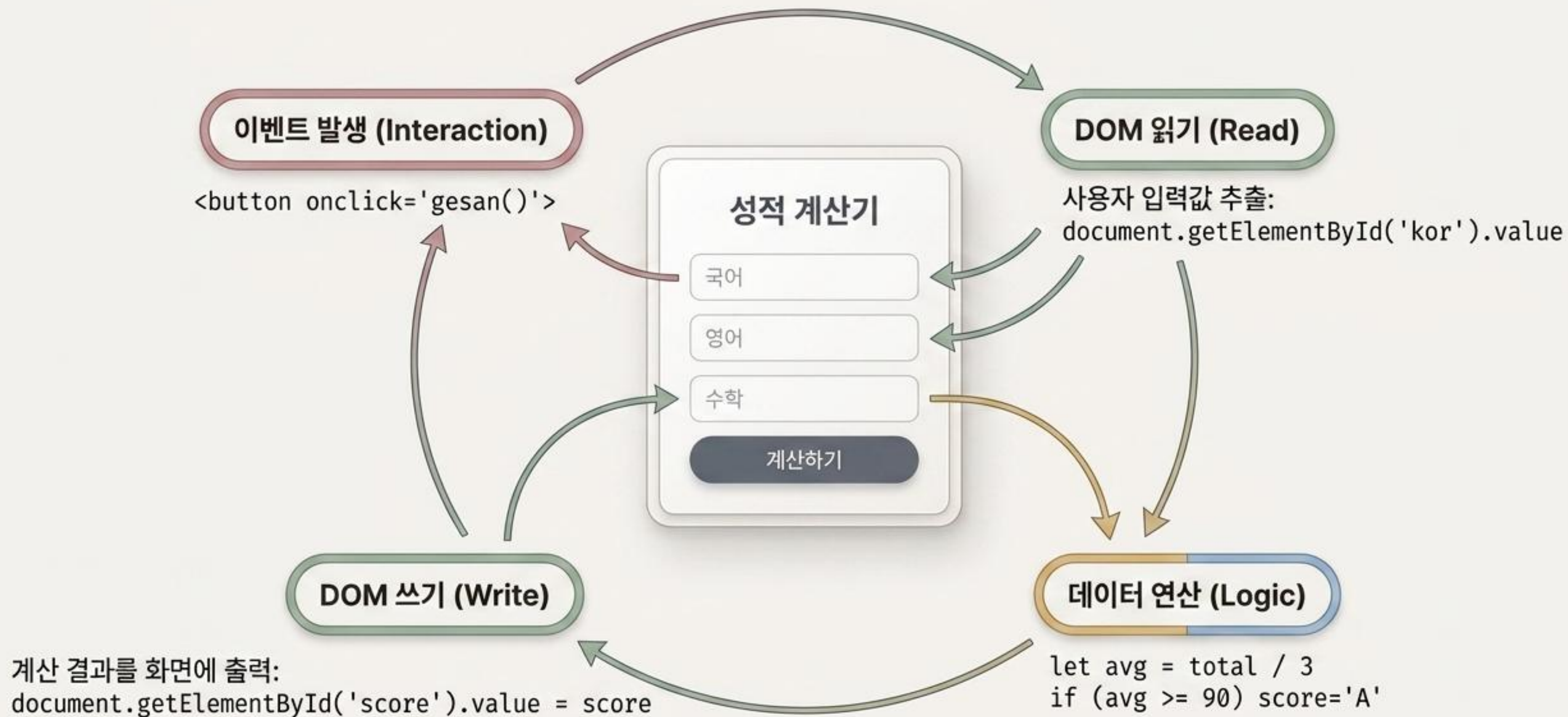
querySelector('CSS선택자')

`<div#panel .action ==>`
`<button='{'.button'>`
CSS 논리(#id, .class)를 사용하여 타겟팅하는 가장 강력한 최신 표준 무기. 다중 선택 시 querySelectorAll() 사용.

DOM 조작: 화면을 바꾸는 3가지 마법



실전 해부: 성적 계산기 아키텍처 통합



JS Core Master Cheatsheet (한 장 요약본)

Functions (함수)

```
function add(a,b) { return a+b }
```

```
let add = function(a,b) { return a+b }
```

```
let add = (a,b) => a+b
```

Data (배열과 객체)

배열: `arr.push()`, `arr.pop()`, `arr.slice()`

객체: `obj.key`, `obj['key']`

순회: `for(let key in obj)`

Events (이벤트)

고전: `tag.onclick = func`

모던: `tag.addEventListener('click', func)`

DOM Manipulation (문서 제어)

선택: `document.querySelector('#id')`

다중 선택: `document.querySelectorAll('.class')`

조작: `tag.innerHTML = '문자'`,
`tag.style.color = 'red'`, `tag.value`


```
{  
  "key": "value",  
  "array": [1, 2, 3],  
  "nested": {  
    "data": true  
  }  
}
```

```
(a, b) => a + b
```

```
const process = async () => {  
  await fetch();  
}
```

JavaScript Core Blueprint

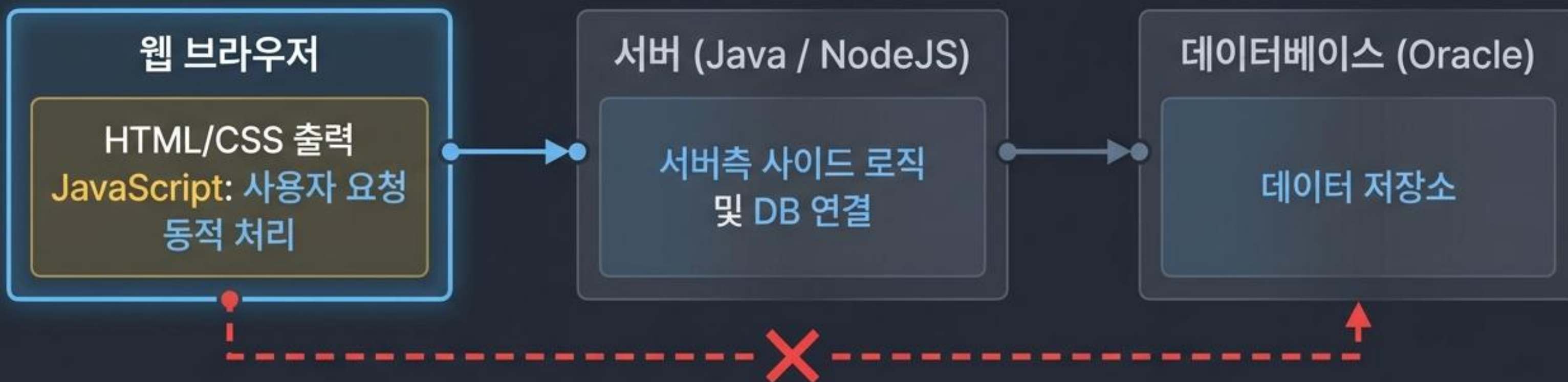
정적 웹에서 동적 애플리케이션으로:
정적 웹에서 동적 애플리케이션으로:

브라우저 동작 원리부터 DOM 제어까지

```
const process = async () => {  
  await fetch();  
}
```

```
const process = async () => {  
  await fetch();  
}
```


웹 생태계 속 자바스크립트의 고립과 독립



브라우저 내부의 자바스크립트는 오라클(DB)에 직접 연결할 수 없습니다.
오직 브라우저 내에서 데이터를 동적으로 가공하고 화면을 다시 그리는(View) 역할에 집중합니다.

실행 환경의 차이: 컴파일러 vs 인터프리터

Java (컴파일 실행)

소스코드

```
static Asr() {  
    System.out.println("..");  
}
```

컴파일러

기계어

한 번에 출력
(사전 에러 검출 용이)

JavaScript (인터프리터 실행)

소스코드

```
function() {  
    System.log("\n", "");  
}
```

인터프리터



한 줄씩 즉시 실행



JS는 한 줄씩 읽어 실행하므로 에러 처리가 까다롭습니다.
화면에 아무것도 안 뜨는 '흰 화면' 오류 시,
반드시 **브라우저의 개발자 도구(Console)**를 통해
`console.log()`로 추적해야 합니다.

변수 선언의 진화: 왜 let과 const를 써야 하는가?

var (ES5) ⚠️

스코프(사용 범위) 불명확.

자동 지정되며
함수 지역변수로만 작용.

사용 지양.

let (ES6) ✓

재할당 가능.

블록 {}'이 종료되면
메모리 자동 회수.

모던 JS의 표준.

const (ES6) 🔒

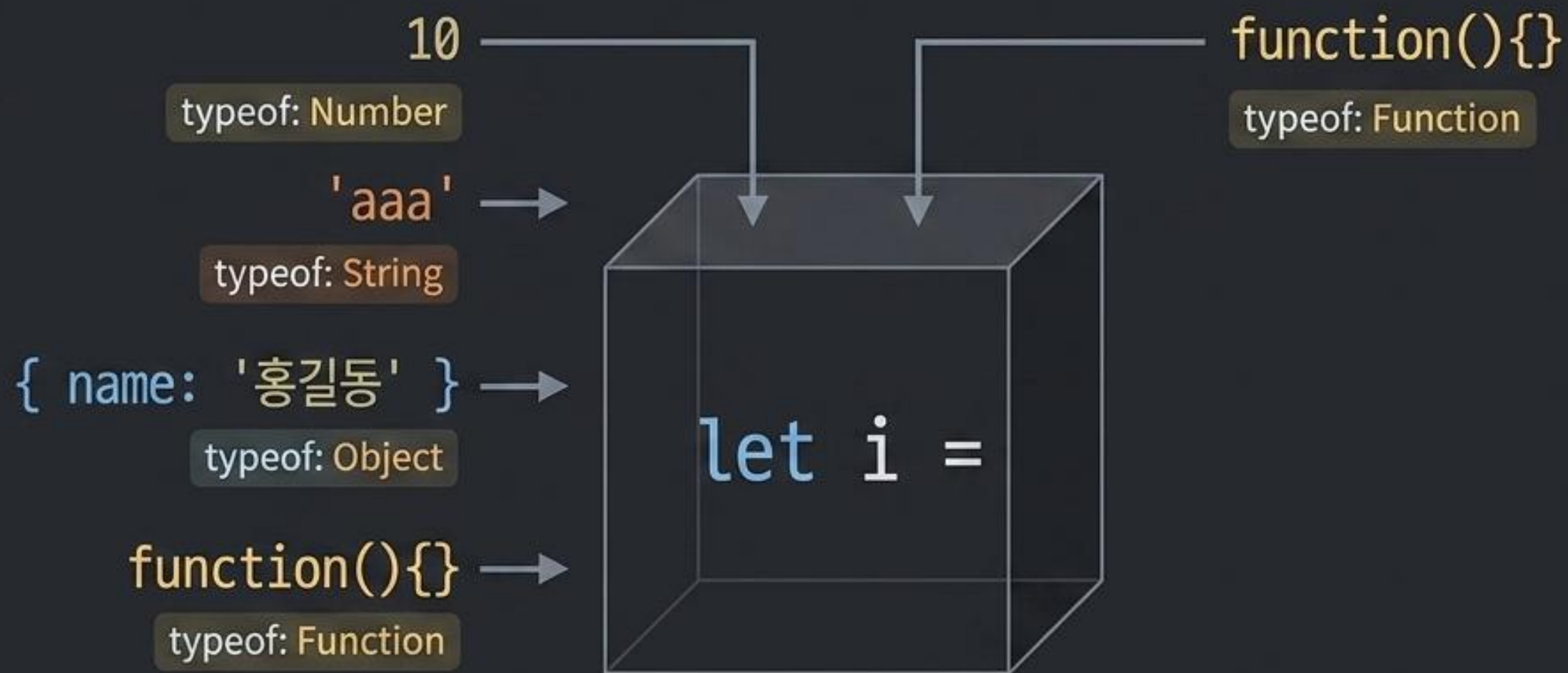
재할당 불가능.

상수형 변수
(Java의 final 역할).

데이터 보호.

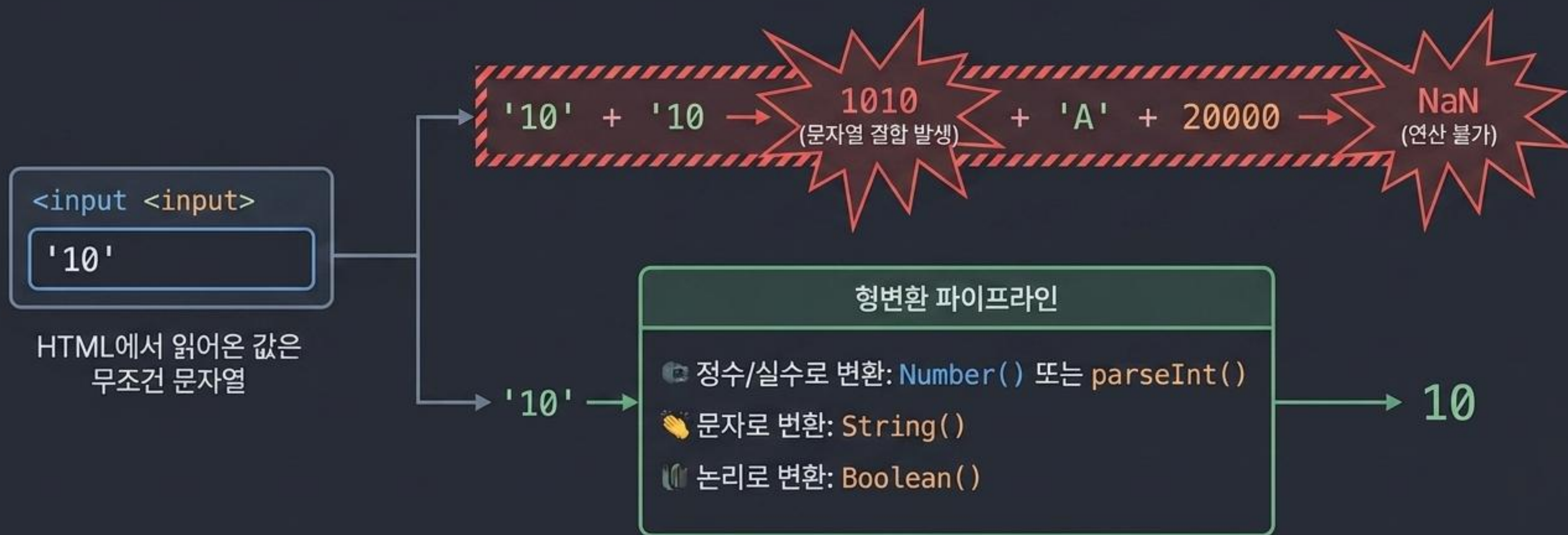
기본적으로 **const**를 사용하고, 값이 변경될 여지가 있는 데이터에만 **let**을 사용하세요.

유연하지만 위험한 '동적 타이핑(Dynamic Typing)'



자바스크립트는 선언 시 데이터형(int, String)을 강제하지 않으며,
들어가는 값에 따라 타입이 자동으로 변합니다.
이로 인한 가독성 저하와 런타임 에러를 막기 위해 TypeScript가 등장했습니다.

데이터 타입의 함정: 10 + 10 = 1010?



Truthy & Falsy: 자바스크립트의 논리 판별법

Falsy 값 (외워야 할 예외들)

- 숫자 `0`, `0.0`
- 값이 없는 상태 `null`
- 선언만 된 상태 `undefined`
- 빈 문자열 `''`

Truthy 값 (나머지 모두)

- 나머지 모든 숫자 (예: `1`, `-1`)
- 모든 문자열 (예: `'Hello'`, 심지어 `'false'`라는 문자열도 `True`)
- 모든 객체와 배열 `{}`, `[]`

엄격한 연산자 통제와 코드 압축



느슨한 비교 (==)

`'10' == 10`

값만 비교. (결과: True). **사용 지양.**



엄격한 비교 (===)

`'10' === 10`

값과 타입 모두 비교. (결과: False).
모던 JS 표준.

코드 압축

Before

```
if (6 % 2 === 0) {  
  let g = '짝수';  
} else {  
  let g = '홀수';  
}
```

After

```
let g = (6 % 2 === 0) ? '짝수' : '홀수';
```

(조건) ? True일 때 값 : False일 때 값
구조로 제어문을 1줄로 압축.

함수의 진화: 화살표 함수 (Arrow Functions)

ES5 일반 함수

Code Diet

```
let a = function(name) {  
    return 'Hello ' + name;  
}
```

function 키워드 제거

function 키워드 제거

```
return 'Hello ' + name;  
}
```

return과 {} 생략
(단일 실행문)

return과 {} 생략 (단일 실행문)

```
let a = (name) => 'Hello ' + name;
```

콜백(Callback)이나 반복문 내부에서 함수형 프로그래밍을 할 때
코드를 획기적으로 줄여줍니다.

데이터 구조 매칭 (Java ↔ JavaScript)

서버 환경 (Java)



```
class Sawon {  
    int sabun;  
    String name;  
}
```

브라우저 환경 (JavaScript)



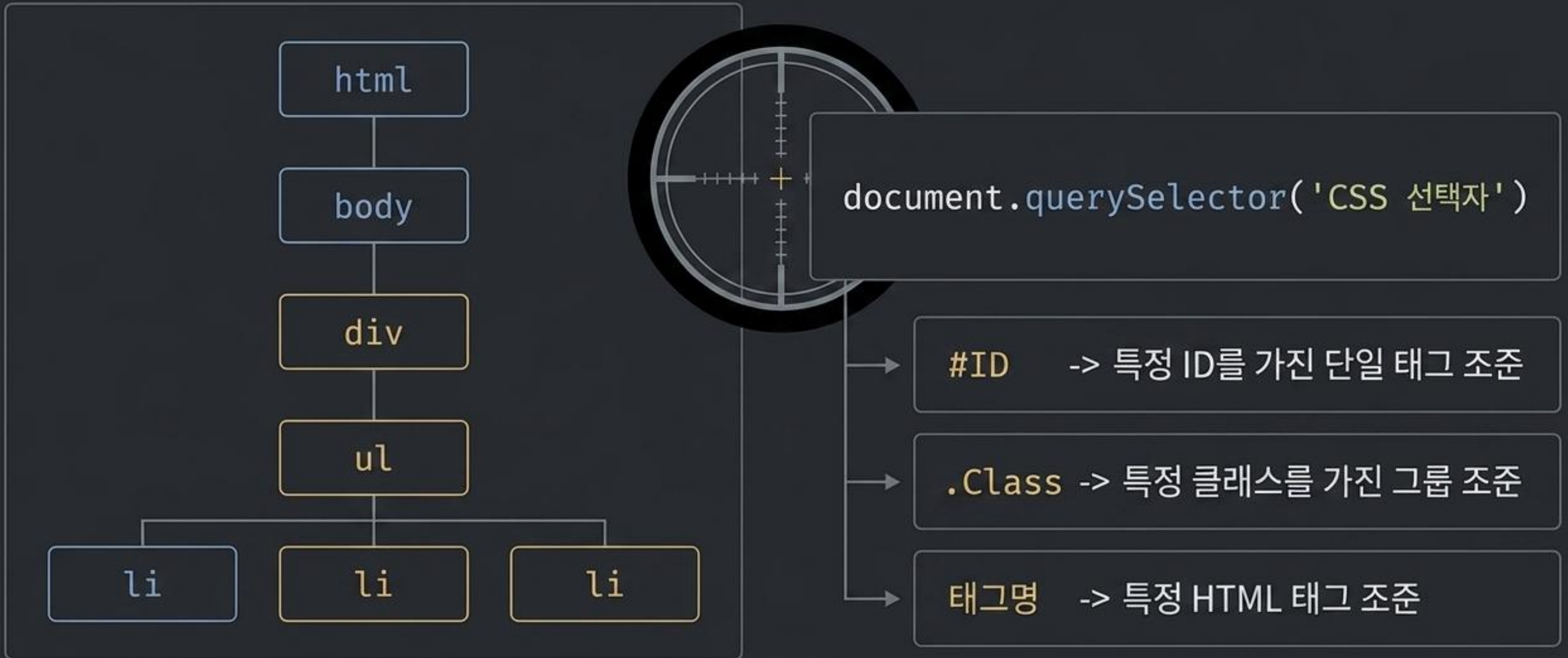
```
{  
    sabun: 1,  
    name: '홍길동'  
}
```

서버의 List<VO>는 브라우저로 전송될 때
[{ JSON }, { JSON }] 형태의 객체 배열로 매칭되어 통신합니다.

반복문 생태계 (The Iteration Matrix)

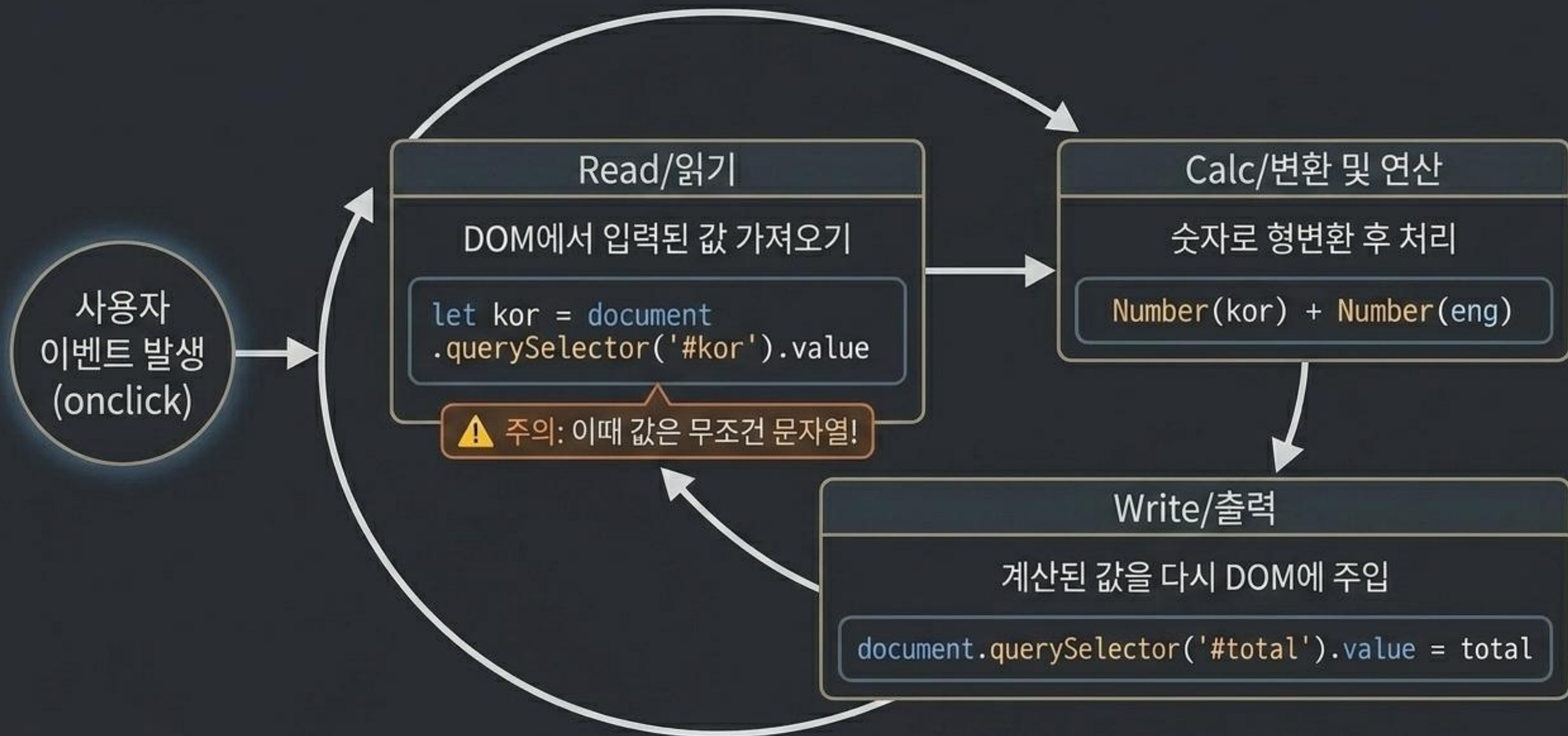


DOM 타겟팅: HTML 요소를 정확히 찾아내는 법



요소를 정확히 선택(Select)해야만 이벤트를 부여하거나 데이터를 주입할 수 있습니다.

미니 앱 사이클: Read → Calc → Write



The Master Pipeline: Data to View

원자재 - Data

JSON 배열 데이터

```
const sawons = [  
  {sabun: 1,  
    name: '홍길동'},  
  ...]
```

조립 - Iterate & Assemble

반복문과 문자열 결합

```
forEach / map 가동  
html += '<li>' +  
  sawon.name +  
  '</li>'
```

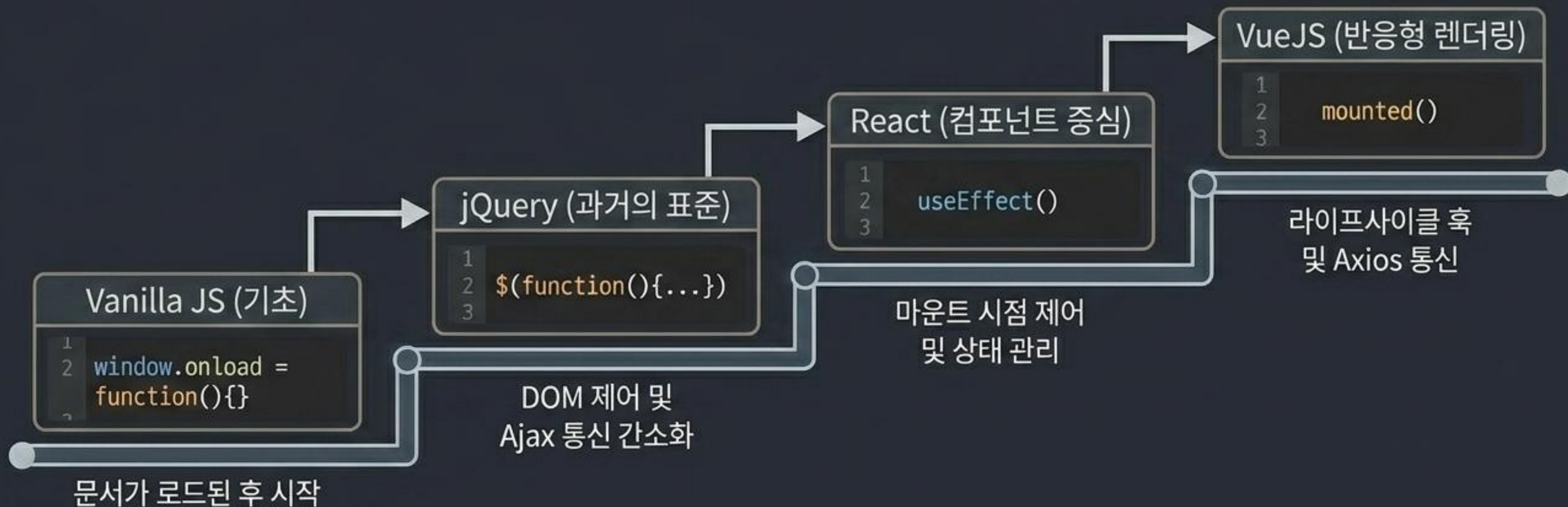
출력 - Render

DOM 주입

```
document.  
  .querySelector('ul')  
  .innerHTML = html;
```

데이터(JSON)를 반복문으로 돌려 HTML 텍스트 블록을 조립하고,
한 번에 DOM에 꽂아 넣는 것이 동적 웹 렌더링의 코어 메커니즘입니다.

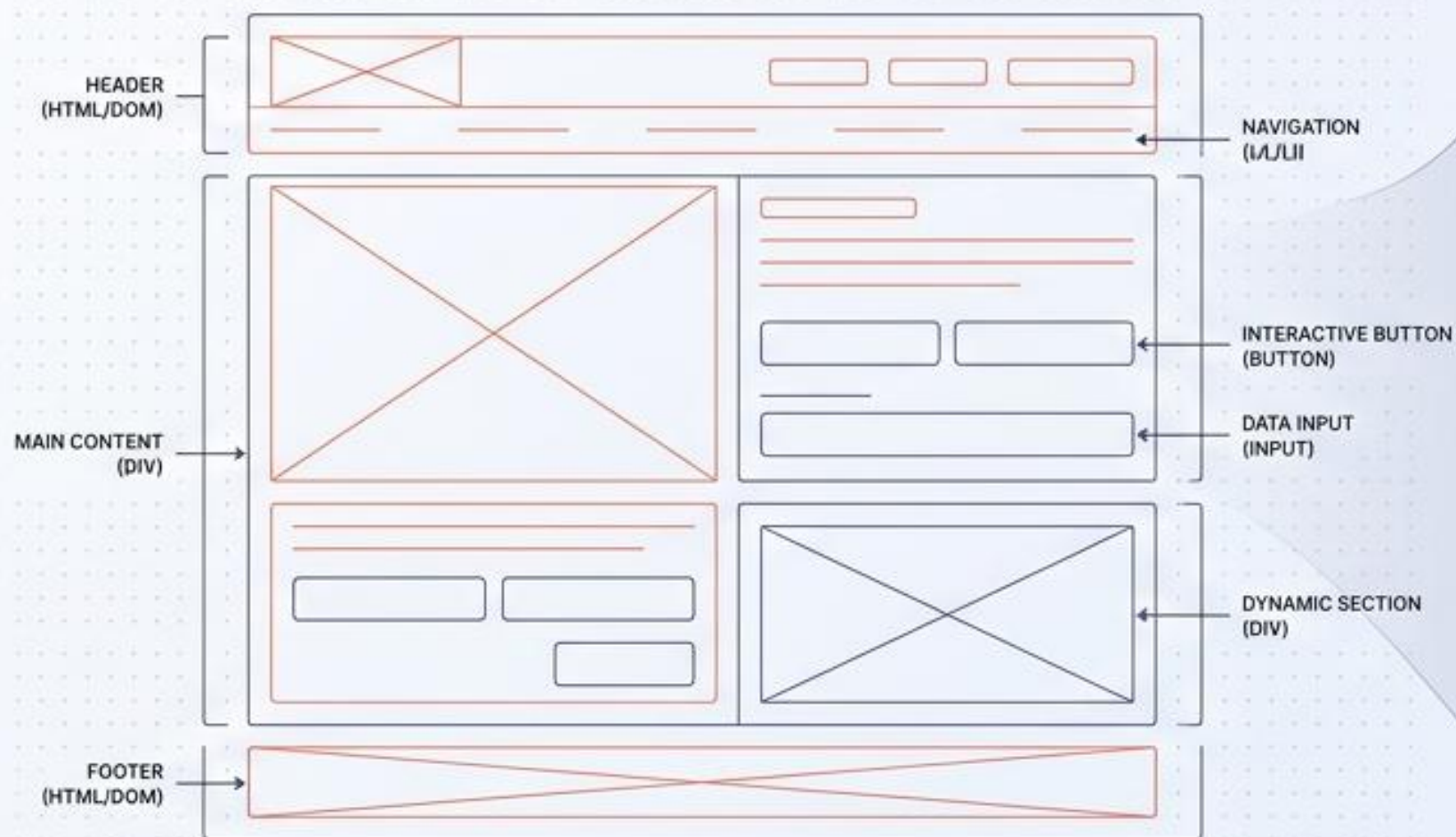
진화하는 생명주기: 프레임워크로의 여정



변수, 타입, 연산, 루프, DOM 제어라는 오늘의 코어 지식은
어떤 프레임워크를 만나든 변하지 않는 강력한 기반(Blueprint)이 될 것입니다.

자바스크립트 마스터리: 정적 웹에서 동적 웹으로

핵심 문법, 데이터 구조, 그리고 DOM 제어의 완벽 해설



```
<div id="app">
  <header>
    <nav>
      <ul>
        <li><a href="#">Home</a></li>
        <li><a href="#">About</a></li>
        <li><a href="#">Contact</a></li>
      </ul>
    </nav>
  </header>
  <main>
    <section id="hero">
      <h1>Welcome to Dynamic Web</h1>
      <button id="interactive-btn">Click Me</button>
    </section>
    <section id="data-display">
      <!-- Dynamic Content Will Appear Here -->
    </section>
  </main>
</div>
```

```
// JavaScript Logic
const app = document.getElementById('app');
const interactiveBtn = document.getElementById('interactive-btn');
const dataDisplay = document.getElementById('data-display');

let clickCount = 0; // Variable state ⚠️ Mutable State: Handle with Care

// Function to handle button click
interactiveBtn.addEventListener('click', () => {
  clickCount++;
  dataDisplay.innerHTML = `
    <p class="success-message">Button clicked ${clickCount} times!</p>
    <p>New content dynamically added to the DOM.</p>
  `;
  dataDisplay.classList.add('active');

  // Example of data structure and manipulation
  const users = [
    { id: 1, name: 'Alice', role: 'Admin' },
    { id: 2, name: 'Bob', role: 'Editor' }
  ];
  const userList = document.createElement('ul');
  users.forEach(user => {
    const listItem = document.createElement('li');
    listItem.textContent = `${user.name} (${user.role})`;
    userList.appendChild(listItem);
  });
  dataDisplay.appendChild(userList);

  console.log('DOM updated successfully with new data.');// ⚡ Dynamic DOM Update
});
```


웹 생태계의 분업 (Static vs. Dynamic)

정적 (Static)



HTML/CSS는 브라우저에 고정된 화면 구조와 스타일을 담당합니다.

동적 (Dynamic)

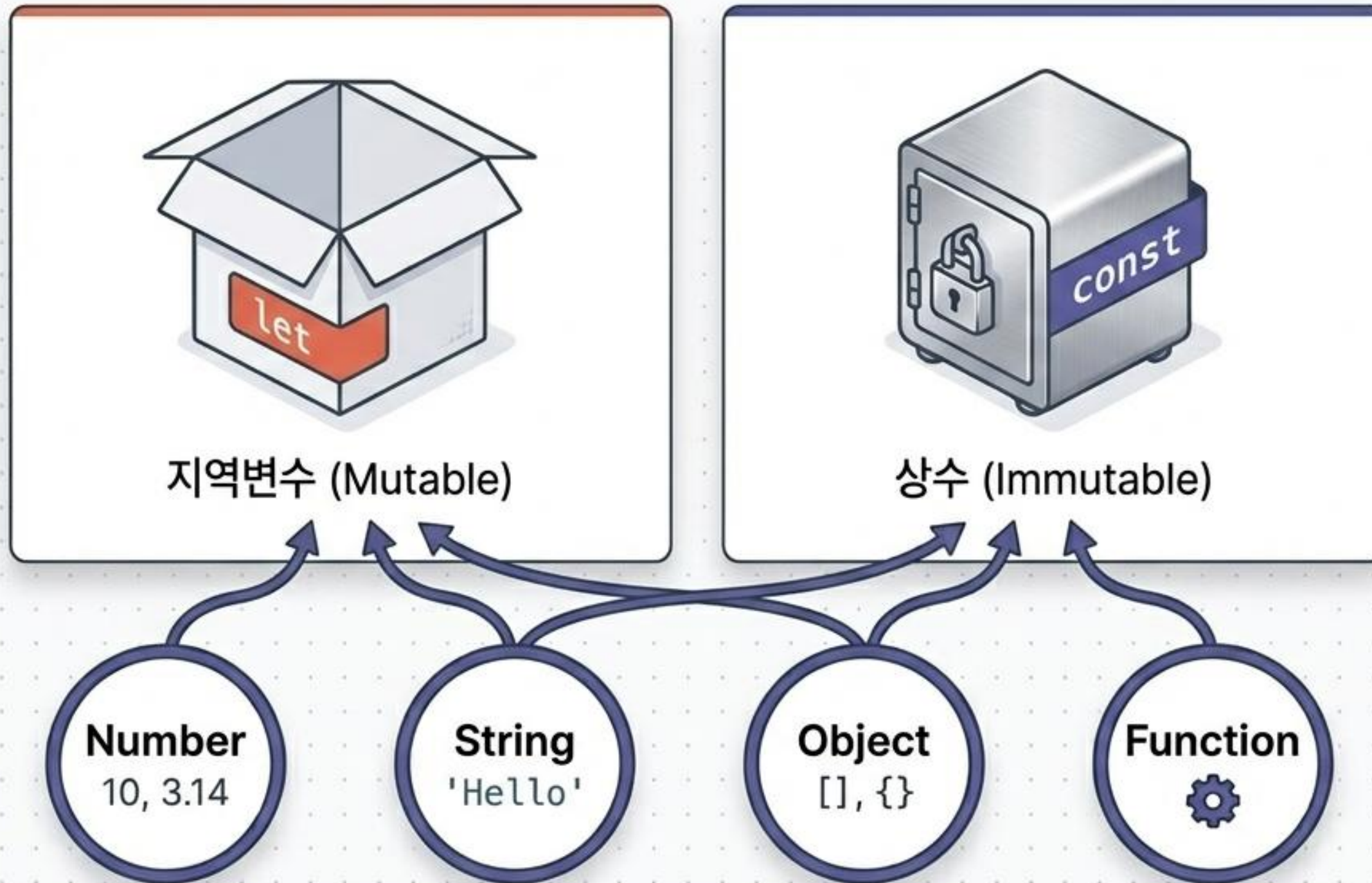


JavaScript는 브라우저 내 화면 출력의 실시간 변경과 서버와의 비동기 통신을 담당합니다.



주의: HTML 폼에서 읽어온 모든 입력 값은 초기 상태에서 무조건 **String(문자열)**으로 취급됩니다.

코어 빌딩 블록: 변수와 데이터형



자바스크립트의 특징:
대입 값에 따라 데이터형이
자동 변경됨

```
const num = () {  
  lo: "10";  
  const.typeof "num");  
  
typeof num () {  
  const.typeof "{}");  
}
```

typeof 연산자를 통해
현재 데이터형 확인 가능

자바스크립트의 엄격성: 연산자와 형변환

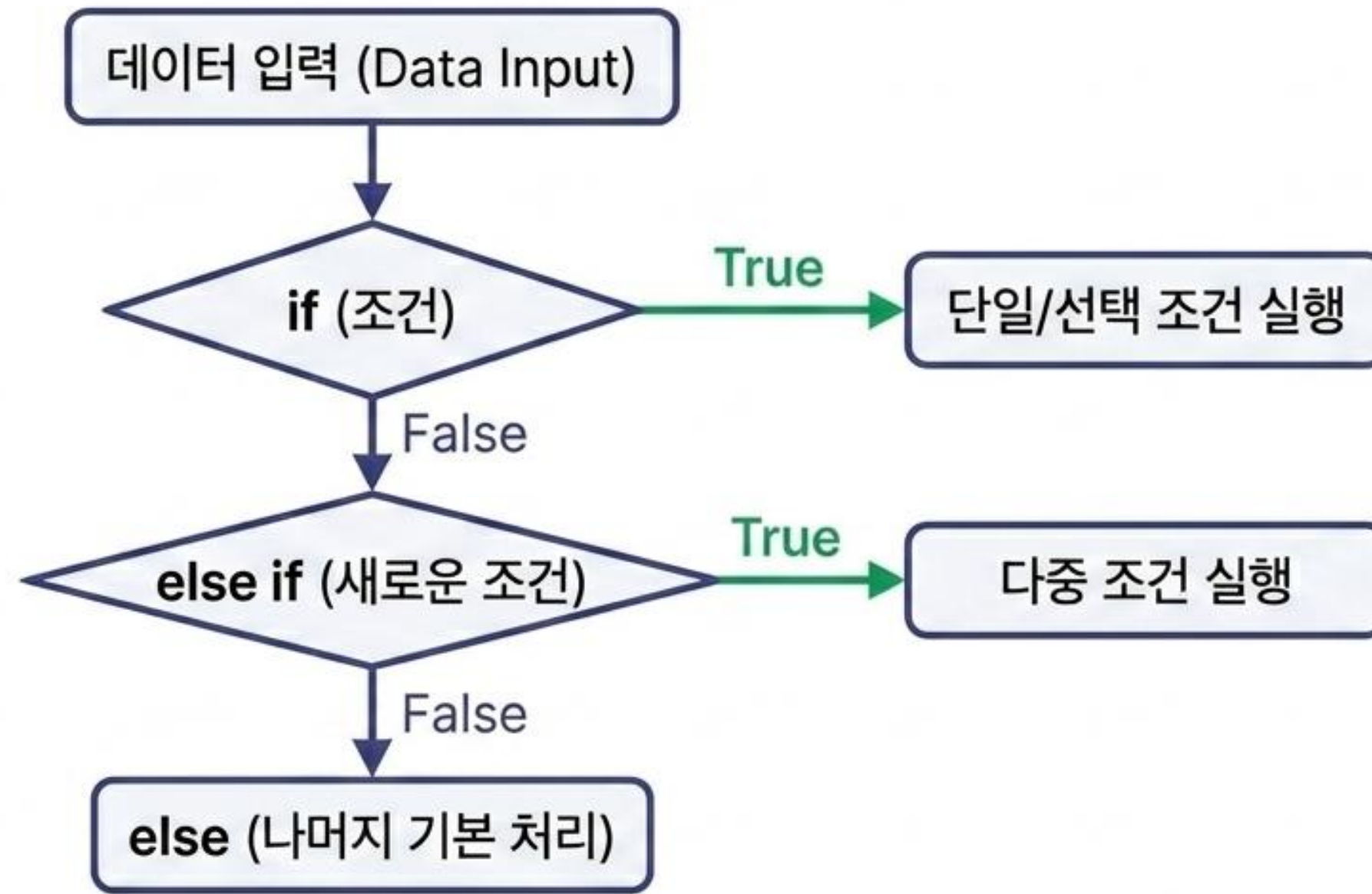
	느슨한 비교
==	<code>5 == '5' → True</code> 자동 형변환 발생
	엄격한 비교
===	<code>5 === '5' → False</code> VueJS/ReactJS 환경에서 필수적인 권장 사항

형변환 도구

- 숫자 변환: `Number()`, `parseInt()`
- 문자 변환: `String()`
- 논리 변환: `Boolean()`

⚠ Diagnostic Rule: 0, 0.0, null이 아닌 모든 수는 true로 판별됨. (예: `Boolean(1) -> true`)

제어문: 논리적 의사결정 트리



반복문 매트릭스 (Mastering Loops)

구문 (Syntax)	대상 (Target)	특징 (Key Feature)
for	일반적 반복	조건식 기반, 중간 중단 가능
for-in	배열의 인덱스 / 객체의 키	0, 1... 또는 객체의 속성(key) 이름 읽기
for-of	배열의 실제 값	배열에 저장된 실제 데이터(value) 순회
forEach	배열 전용 순회	콜백 함수를 통한 값 순회 (배열.forEach(function(값){}))
map ★★★★★	배열 전용, 새로운 배열 반환	콜백 함수 실행 후 새로운 데이터 배열 생성. React 등 최신 프레임워크 필수 요소.

함수의 진화: 선언적 함수에서 화살표 함수로

선언적 함수

```
function plus(a, b) { return a+b }
```

익명 함수

```
let plus = function(a, b) { return a+b }
```

화살표 함수

```
let plus = (a, b) => { return a+b }
```

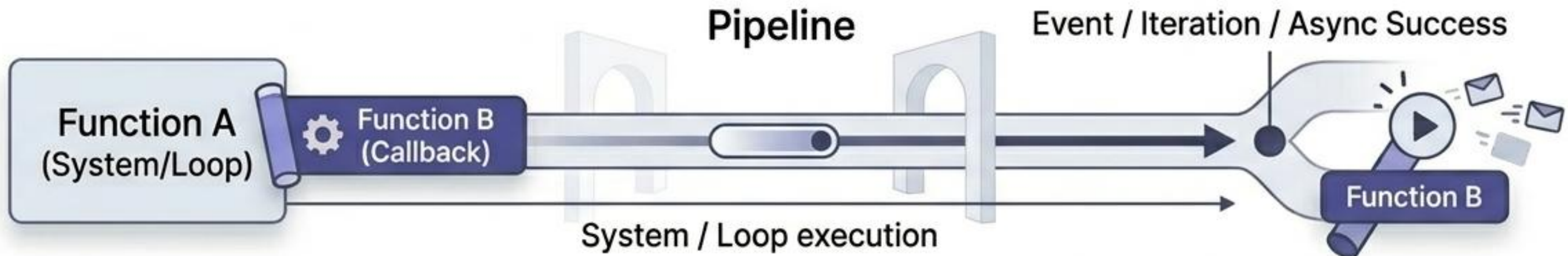
람다식 축약 - 권장

```
let plus = (a, b) => a + b
```

가독성을 극대화한
현대적 권장 방식

콜백 (Callback) 함수의 힘

매개변수로 전송되어 시스템에 의해 '자동 호출'되는 함수



1. 배열 반복 제어

```
map(function(){}),  
forEach(function(){})
```

2. 비동기 서버 응답

```
then(function(response){},  
ajax success:function(){})
```

데이터 패키징 1: 배열(Array)과 제어 메서드

... (Spread) - 배열 복사 및 확장



```
let names = ['홍길동', '심청이']
```



push('이순신') - 데이터 추가



slice(1, 2) - 범위 자르기



pop() / shift() - 데이터 삭제

참고: 자바스크립트의 배열은 혼합 데이터형 저장 가능하며, length 속성으로 크기를 확인합니다.

데이터 패키징 2: 객체(Object)와 JSON



```
{  
  sabun: 1,  
  name: '홍길동',  
  dept: '개발부',  
  pay: 3500  
}
```

데이터 접근: `sawon.name` (점 표기법)
또는 `sawon['name']` (괄호 표기법)

JSON (JavaScript Object Notation)은 서버와 클라이언트 간 데이터 전송의 핵심 표준입니다.

객체 내부 메서드와 this

Encapsulation Blueprint

Object: sawon

Variables/Properties

name: '홍길동'

this

this

Methods/Functions

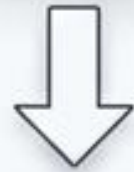
getName: function() {}

setName: function(name) {}

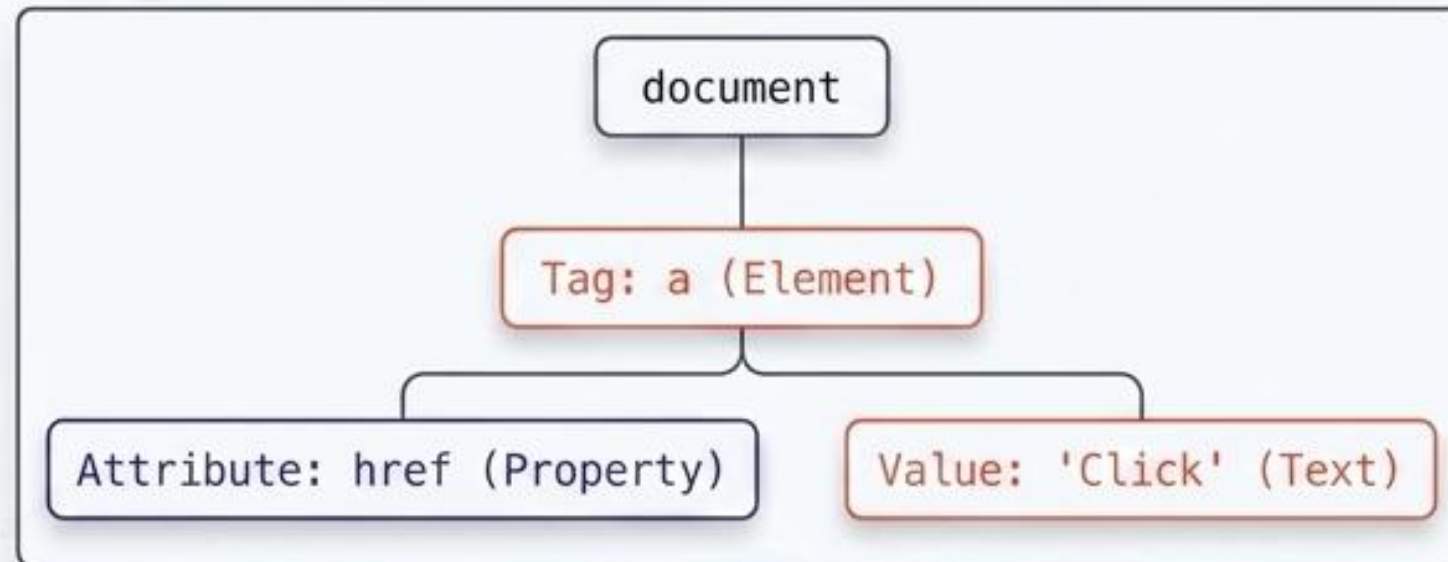
이 구조는 객체가 변수(속성)와 함수(동작)를 동시에 가지는 완결된 구조를 보여줍니다.
Vue, React 등 최신 컴포넌트 기반 프레임워크의 클래스 객체 모델로 직접 연결됩니다.

DOM 트리: HTML과 JS의 연결 다리

```
<a href='url'>Click</a>
```



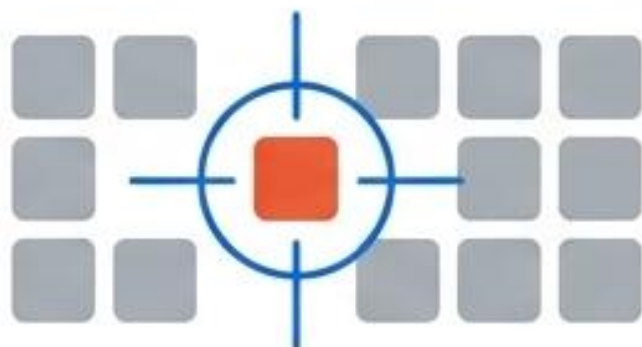
Document Object Model (DOM) 변환



DOM은 브라우저가 HTML을 자바스크립트가 제어할 수 있는 객체로 변환한 구조입니다. 이를 통해 속성값 변경, CSS 적용, 이벤트 처리가 가능해집니다.

DOM 선택 도구 (Selection API)

단일 타겟



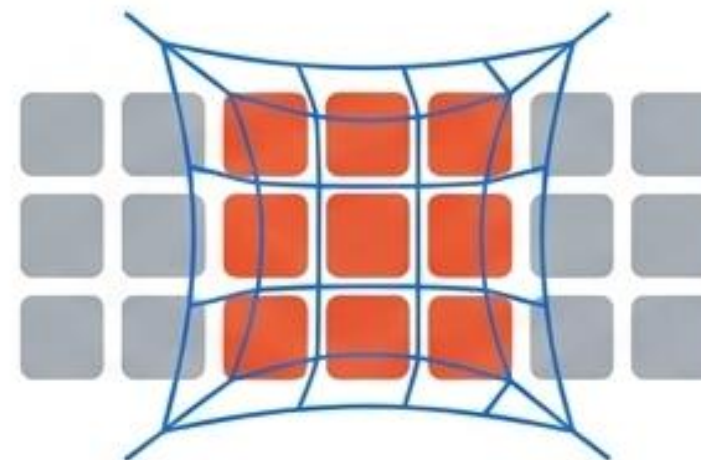
```
document.getElementById('id')
```

```
document.querySelector('#id, .class')
```

권장 표준

```
document.querySelector('#id, .class')
```

다중 타겟 그룹



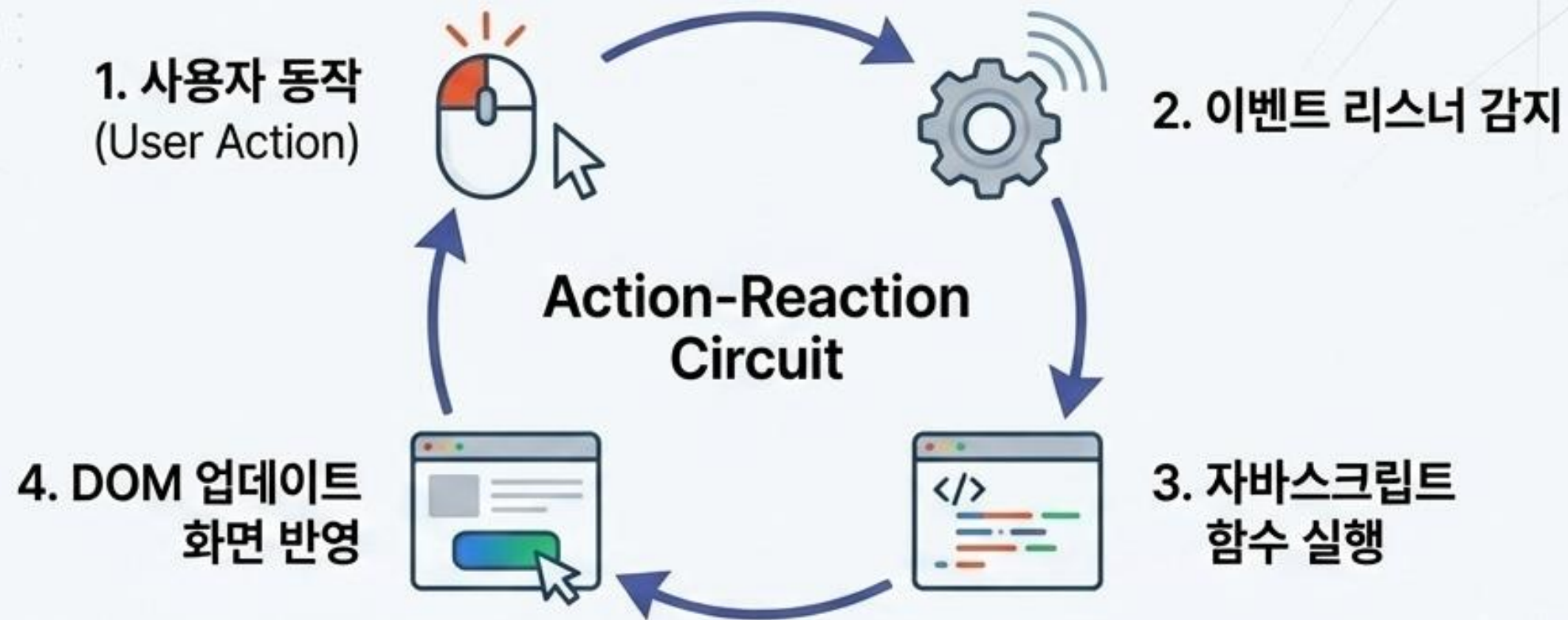
```
document.querySelectorAll()
```

```
document.getElementsByClassName()
```

결과값은 배열(Array/NodeList) 형태로 반환됩니다.

CSS 선택자(Selector) 방식을 그대로 사용하는 querySelector 방식이 현대 웹 개발의 표준입니다.

상호작용의 원리: 이벤트 핸들링



고전 이벤트 방식

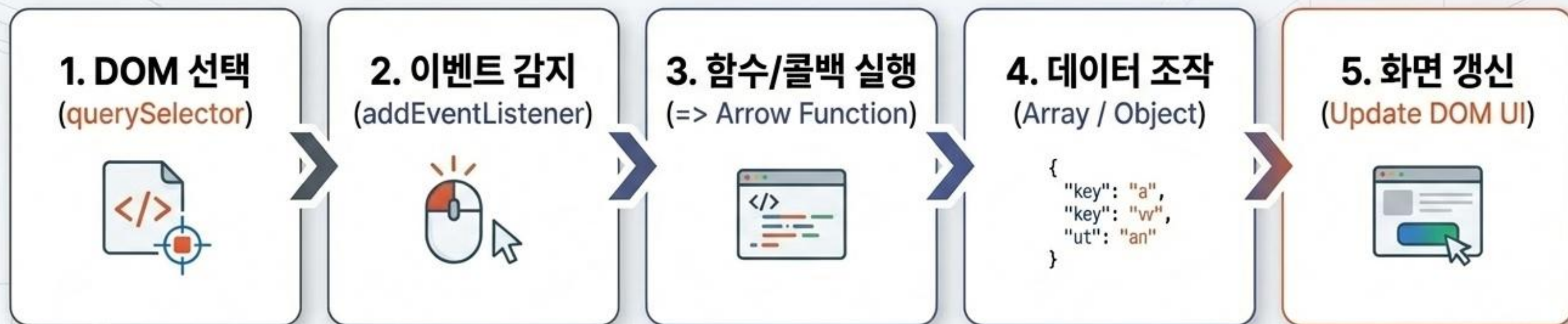
```
img.onclick = function() { ... }
```

모던 리스너 방식 (권장)

```
btn.addEventListener('click', function() { ... })
```

주요 이벤트: `click`, `mouseover`, `mouseout`, `mousedown`, `mouseup`, `keydown`

총정리: 모던 자바스크립트 워크플로우



정적 문서를 동적 애플리케이션으로 변환하는 최종 사이클.
요소를 선택하고, 행위를 감지하여, 데이터를 연산한 뒤 화면을 갱신합니다.

이 워크플로우의 완벽한 이해는 바닐라 자바스크립트를 넘어
React, Vue 등 현대적 프레임워크 환경을 지배하는 가장 확실한 기초가 됩니다.